



Universidad Tecnológica del Centro de Veracruz
Ingeniería en Redes Inteligentes y Ciberseguridad

Práctica General:

NetAdmin API – Sistema de Inventario, Monitoreo y Administración de Red

Materia:

Automatización de Infraestructura Digital I

Docente:

Ing. Michel Orozco Carrera

Integrantes del equipo:

Luz Graciela Torales Rodríguez

Tania Rojas Arévalo

Cesar Gustavo Illescas Dominguez

Grupo:

IRIC 9A

Fecha de entrega:

Martes, 14 Julio del 2026

Tabla de contenido

1. Introducción.....	5
2. Objetivo general	5
3. Objetivos específicos.....	5
4. Alcance del proyecto	6
5. Herramientas utilizadas	6
6. Arquitectura general del sistema	7
7. Descripción general del funcionamiento	8
8. Estructura del proyecto	9
9. Descripción de archivos principales.....	11
10. Preparación del entorno de desarrollo	12
11. Ejecución del servidor FastAPI	12
12. Verificación inicial de la API	13
13. Documentación automática con Swagger	13
14. Repositorio inicial del proyecto	15
15. Desarrollo de la API REST	15
16. Endpoints implementados	16
17. Creación de dispositivos con POST	17
18. Consulta del inventario con GET	18
19. Búsqueda de dispositivo por IP	19
20. Actualización de dispositivos con PUT.....	20
21. Eliminación de dispositivos con DELETE.....	22
22. Validación de CRUD en Postman	23
23. Resultado de la implementación CRUD.....	29
24. Escaneo de red local simulada	29
25. Resultado del escaneo	30
26. Actualización del inventario después del escaneo.....	31
27. Exportación del inventario	32
28. Archivos generados.....	33

29. Formatos de exportación utilizados.....	34
30. Pruebas desde Postman.....	34
31. Resultado de la sección	36
32. Administración remota en NetAdmin API	36
33. Administración Linux mediante Paramiko	36
34. Activación del servicio SSH en Linux	37
35. Prueba de conexión SSH desde Windows	38
36. Ejecución de comando con Paramiko desde la API	39
37. Ejecución de comandos administrativos en Linux	40
38. Administración de switch/router mediante Netmiko	42
39. Ejecución de comando show ip interface brief	43
40. Consulta de VLANs mediante Netmiko	44
41. Prueba unitaria del endpoint Netmiko simulado.....	45
42. Comparación entre Paramiko y Netmiko	46
43. Resultado de la administración remota	46
44. Pruebas unitarias con Pytest	47
45. Ejecución de Pytest.....	48
46. Importancia de las pruebas unitarias	49
47. Validación de códigos HTTP	49
48. Error 400: IP duplicada	50
49. Error 404: dispositivo no encontrado	51
50. Error 422: cuerpo JSON incompleto.....	52
51. Error 500: fallo de conexión con Paramiko.....	53
52. Error 500: fallo de conexión con Netmiko.....	54
53. Tabla de diagnóstico de códigos HTTP	55
54. Análisis de resultados de la validación HTTP	56
55. Resultado de la sección	57
56. Uso de GitHub como repositorio del proyecto	57
57. Organización del trabajo mediante issues.....	59

58. Uso de ramas de trabajo.....	59
59. Pull request para integrar cambios	60
60. Historial de commits.....	62
61. Evidencias de trabajo colaborativo	62
62. Resultados generales del proyecto.....	63
63. Cumplimiento de requisitos de la práctica.....	63
64. Conclusiones	64
65. Recomendaciones y mejoras futuras	65
66. Anexos	66
67. Cierre del reporte.....	66

1. Introducción

El presente reporte documenta el desarrollo de **NetAdmin API**, un sistema de inventario, monitoreo y administración básica de red desarrollado en Python mediante el framework **FastAPI**. El propósito principal de esta práctica fue construir una aplicación capaz de registrar dispositivos de red, consultar información del inventario, actualizar registros, eliminar dispositivos obsoletos, escanear una red local o simulada y exportar la información en formatos estructurados.

Durante el desarrollo del proyecto se implementaron diferentes endpoints REST para realizar operaciones CRUD sobre dispositivos de red, además de funciones para escaneo básico, exportación de inventario, administración remota mediante **Paramiko** y **Netmiko**, pruebas unitarias con **Pytest** y validación de códigos de respuesta HTTP.

El sistema fue probado mediante la documentación automática de FastAPI en /docs y posteriormente mediante **Postman**, con el objetivo de documentar evidencias gráficas del funcionamiento de cada endpoint. También se utilizó **GitHub** como repositorio del proyecto y como medio para evidenciar el trabajo colaborativo mediante ramas, issues, commits y pull requests.

2. Objetivo general

Desarrollar una aplicación en Python que permita recabar información básica de una red, almacenarla en formatos estructurados y administrarla mediante scripts y una API REST.

3. Objetivos específicos

- Implementar una API REST funcional que permita registrar, consultar, actualizar y eliminar dispositivos del inventario.
- Realizar un escaneo básico de una red local o simulada para detectar dispositivos activos.
- Mantener un inventario actualizado con información como dirección IP, hostname, dirección MAC, tipo de dispositivo, sistema operativo aproximado y estado.

- Exportar la información del inventario en formatos JSON, YAML y XML.
- Probar los endpoints principales mediante Swagger y Postman.
- Validar códigos de estado HTTP generados por operaciones exitosas y errores controlados.
- Implementar módulos de administración remota mediante Paramiko para equipos Linux y Netmiko para equipos de red tipo Cisco.
- Documentar el proyecto en GitHub mediante README, ramas, issues, commits y pull requests.

4. Alcance del proyecto

El proyecto fue desarrollado para funcionar en un entorno local o simulado. Para las pruebas de escaneo se utilizó una red controlada, evitando realizar exploraciones sobre redes externas o no autorizadas.

La red utilizada durante la práctica fue:

192.168.163.0/28

El sistema administra únicamente dispositivos detectados mediante el escaneo o registrados manualmente a través de la API. La información del inventario se mantiene en memoria durante la ejecución del servidor y puede exportarse en archivos estructurados dentro de la carpeta data.

Aunque la práctica contempla tecnologías como Netmiko y Paramiko, para el caso de Netmiko se integró una simulación de switch Cisco con el fin de validar el endpoint /red/comando sin depender de un dispositivo físico. Para Paramiko se utilizó una máquina Linux simulada con servicio SSH activo.

5. Herramientas utilizadas

Tabla 1. Herramientas utilizadas en el desarrollo de NetAdmin API.

Herramienta	Uso dentro del proyecto
Python 3.12	Lenguaje principal de desarrollo del sistema.
FastAPI	Framework utilizado para crear la API REST.
Uvicorn	Servidor ASGI utilizado para ejecutar la API localmente.
PyYAML	Librería utilizada para exportar el inventario en formato YAML.

dicttoxml	Librería utilizada para exportar el inventario en formato XML.
JSON	Formato estructurado utilizado para almacenar y responder datos.
Paramiko	Librería utilizada para conexión SSH con equipos Linux.
Netmiko	Librería utilizada para administración de equipos de red tipo Cisco.
Pytest	Herramienta utilizada para realizar pruebas unitarias.
Postman	Herramienta utilizada para probar y documentar los endpoints.
Git	Sistema de control de versiones utilizado durante el desarrollo.
GitHub	Plataforma utilizada para alojar el repositorio y evidenciar trabajo colaborativo.
Visual Studio Code	Editor utilizado para programar y organizar el proyecto.
VMware/Kali Linux	Entorno utilizado para simular pruebas de red y conexión SSH.

6. Arquitectura general del sistema

La arquitectura de **NetAdmin API** se basa en una API REST construida con FastAPI. El usuario interactúa con el sistema mediante Swagger o Postman, enviando solicitudes HTTP a los diferentes endpoints. Dependiendo de la operación solicitada, la API ejecuta módulos internos encargados del inventario, escaneo, exportación o administración remota.

El módulo de inventario administra los dispositivos registrados. El módulo de escaneo permite identificar equipos activos dentro de una red local simulada. El módulo de exportación genera archivos en formato JSON, YAML y XML. Por otra parte, los módulos de administración remota permiten ejecutar comandos en equipos Linux mediante Paramiko y en equipos de red tipo Cisco mediante Netmiko.

La comunicación general del sistema se puede representar de la siguiente manera:

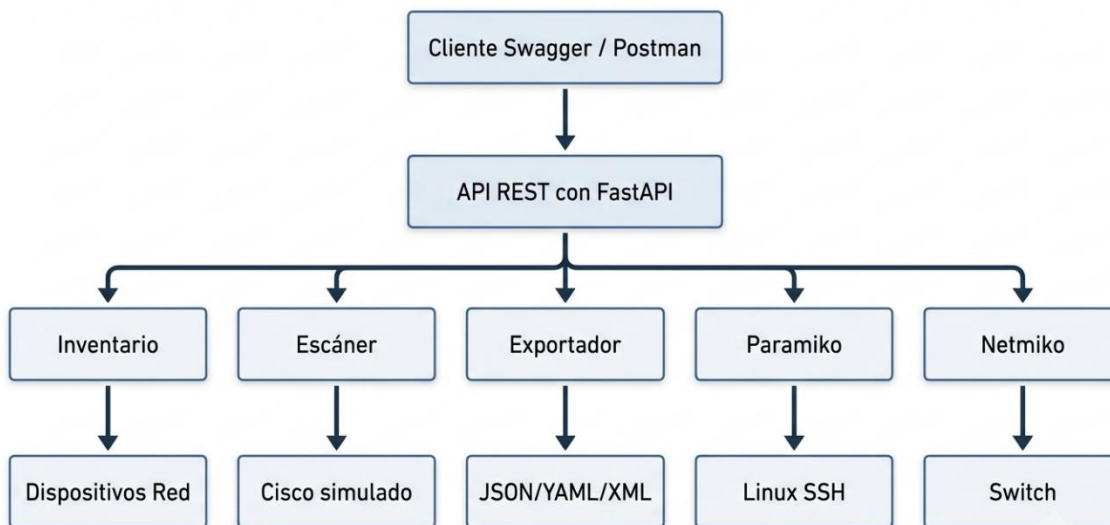


Figura 1. Arquitectura general de funcionamiento de NetAdmin API.

7. Descripción general del funcionamiento

El funcionamiento de NetAdmin API inicia cuando el usuario realiza una solicitud desde Swagger o Postman. Si la solicitud corresponde a una operación CRUD, la API consulta el módulo de inventario para agregar, listar, buscar, actualizar o eliminar dispositivos.

Cuando el usuario solicita un escaneo de red, el sistema utiliza el módulo `escaner.py`, el cual recorre las direcciones IP de una red indicada y realiza pruebas de conectividad para identificar dispositivos activos. Los dispositivos encontrados son agregados al inventario.

Cuando el usuario solicita una exportación, el módulo `exportador.py` toma la información actual del inventario y genera archivos en los formatos `inventario.json`, `inventario.yaml` y `inventario.xml`.

Para la administración remota, el endpoint `/linux/comando` utiliza Paramiko para conectarse por SSH a un equipo Linux y ejecutar comandos administrativos. Por su parte, el endpoint `/red/comando` utiliza Netmiko o una simulación de equipo Cisco para validar comandos como `show ip interface brief` y `show vlan brief`.

Esta arquitectura permite separar las responsabilidades del sistema y facilita su mantenimiento, prueba y documentación.

8. Estructura del proyecto

El proyecto **NetAdmin API** fue organizado en una estructura modular para separar las responsabilidades principales del sistema. Esta organización permite identificar con claridad los archivos encargados de la API, los modelos de datos, el inventario, el escaneo de red, la exportación de información, las conexiones remotas y las pruebas unitarias.

La estructura general del proyecto quedó de la siguiente manera:

netadmin_api/

```
|
|
|└─ app/
|  |└─ __init__.py
|  |└─ main.py
|  |└─ modelos.py
|  |└─ inventario.py
|  |└─ escaner.py
|  |└─ exportador.py
|  |└─ netmiko_admin.py
|  └─ paramiko_admin.py
|
|└─ data/
|  |└─ inventario.json
|  |└─ inventario.yaml
|  └─ inventario.xml
|
|└─ docs/
|  |└─ manual_tecnico.md
```

```
| └─ tabla_http.md
|
|─ tests/
|   └─ test_api.py
|
|─ requirements.txt
|─ README.md
└─ .gitignore
```

La carpeta app contiene los módulos principales de la aplicación. La carpeta data almacena los archivos de inventario exportados en los formatos JSON, YAML y XML. La carpeta tests contiene las pruebas unitarias realizadas con Pytest. La carpeta docs se utiliza para guardar documentación complementaria, como el manual técnico y la tabla de códigos HTTP.

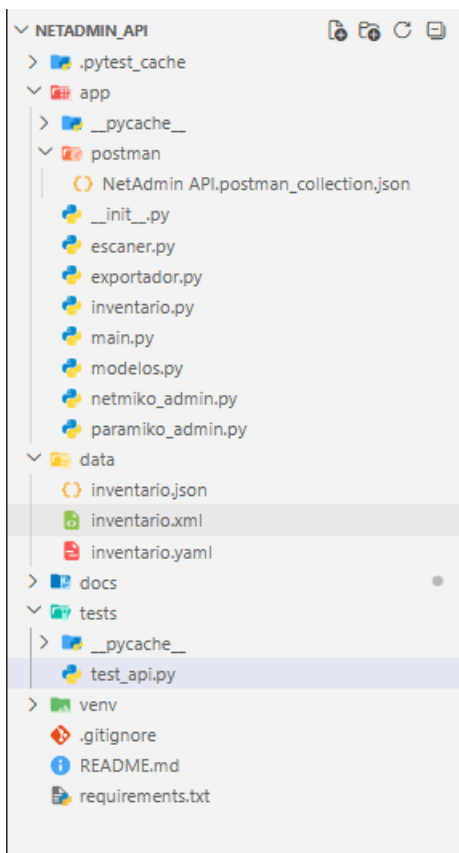


Figura 2. Estructura del proyecto NetAdmin API en Visual Studio Code.

9. Descripción de archivos principales

Para facilitar el mantenimiento del sistema, cada archivo cumple una función específica dentro del proyecto. La siguiente tabla resume el propósito de los archivos más importantes.

Tabla 2. Descripción de los archivos principales del proyecto NetAdmin API.

Archivo o carpeta	Descripción
app/main.py	Archivo principal de la API. Contiene la configuración de FastAPI y los endpoints del sistema.
app/modelos.py	Define los modelos de datos utilizados por la API, como dispositivos, comandos de red y comandos Linux.
app/inventario.py	Administra el inventario en memoria, permitiendo agregar, listar, buscar, actualizar y eliminar dispositivos.
app/escaner.py	Contiene la lógica para escanear una red local o simulada y detectar dispositivos activos.
app/exportador.py	Permite exportar el inventario en formatos JSON, YAML y XML.
app/netmiko_admin.py	Módulo encargado de ejecutar comandos en equipos de red mediante Netmiko o mediante una simulación Cisco.
app/paramiko_admin.py	Módulo encargado de ejecutar comandos en equipos Linux mediante conexión SSH con Paramiko.
data/	Carpeta donde se generan los archivos inventario.json, inventario.yaml y inventario.xml.
tests/test_api.py	Archivo que contiene las pruebas unitarias de la API realizadas con Pytest.
requirements.txt	Archivo que almacena las dependencias necesarias para ejecutar el proyecto.
README.md	Documento principal del repositorio con instrucciones de instalación y uso.
.gitignore	Archivo utilizado para evitar subir carpetas o archivos innecesarios al repositorio, como venv/ o __pycache__.

10. Preparación del entorno de desarrollo

Para desarrollar y ejecutar el proyecto se utilizó un entorno virtual de Python. Esto permite instalar las dependencias necesarias sin afectar la configuración global del sistema operativo.

Primero, se ubicó la terminal en la carpeta principal del proyecto:

```
cd C:\Users\luzgr\Desktop\netadmin_api
```

Posteriormente, se creó el entorno virtual:

```
python -m venv venv
```

Después, se activó el entorno virtual con el siguiente comando:

```
.\venv\Scripts\Activate.ps1
```

En caso de que PowerShell bloqueara la activación del entorno virtual por políticas de ejecución, se utilizó el siguiente comando:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

```
.\venv\Scripts\Activate.ps1
```

Una vez activado el entorno virtual, se instalaron las dependencias principales del proyecto:

```
pip install fastapi uvicorn pyyaml dicttoxml pytest python-multipart netmiko paramiko httpx2
```

Finalmente, se generó el archivo requirements.txt, el cual permite documentar las librerías instaladas y facilitar la instalación del proyecto en otros equipos:

```
pip freeze > requirements.txt
```

11. Ejecución del servidor FastAPI

Con el entorno virtual activo y las dependencias instaladas, se ejecutó el servidor local mediante Uvicorn. Para iniciar la API se utilizó el siguiente comando:

```
python -m uvicorn app.main:app --reload
```

El parámetro `--reload` permite que el servidor se reinicie automáticamente cuando detecta cambios en el código fuente, lo cual facilita el desarrollo y las pruebas.

Al ejecutarse correctamente, la terminal mostró que el servidor estaba disponible en la siguiente dirección:

`http://127.0.0.1:8000`

```
branch 'main' set up to track 'origin/main'.
○ (venv) PS C:\Users\luzgr\Desktop\netadmin_api> python -m uvicorn app.main:app --reload
INFO: Will watch for changes in these directories: ['C:\\Users\\luzgr\\Desktop\\netadmin_api']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [42256] using StatReload
INFO: Started server process [40732]
INFO: Waiting for application startup.
INFO: Application startup complete.
□
```

Figura 3. Ejecución del servidor FastAPI con Uvicorn.

12. Verificación inicial de la API

Después de iniciar el servidor, se realizó una primera validación accediendo a la dirección principal de la API:

`http://127.0.0.1:8000`

El sistema respondió con un mensaje de confirmación indicando que la API estaba funcionando correctamente:

```
{
  "mensaje": "NetAdmin API funcionando correctamente"
}
```

Esta prueba permitió comprobar que FastAPI estaba activo y que el archivo principal `main.py` se cargaba correctamente desde el entorno virtual.

13. Documentación automática con Swagger

FastAPI genera automáticamente una interfaz de documentación interactiva mediante Swagger. Esta documentación permite visualizar todos los endpoints implementados, sus métodos HTTP, parámetros, cuerpos de solicitud y respuestas esperadas.

Para acceder a la documentación automática se utilizó la siguiente URL:

<http://127.0.0.1:8000/docs>

En esta interfaz se observaron los endpoints principales del sistema, incluyendo:

GET /

GET /dispositivos

POST /dispositivos

GET /dispositivos/{ip}

PUT /dispositivos/{ip}

DELETE /dispositivos/{ip}

POST /escanear

GET /exportar

POST /red/comando

POST /linux/comando

La documentación automática fue útil para realizar pruebas iniciales antes de validar los endpoints en Postman.

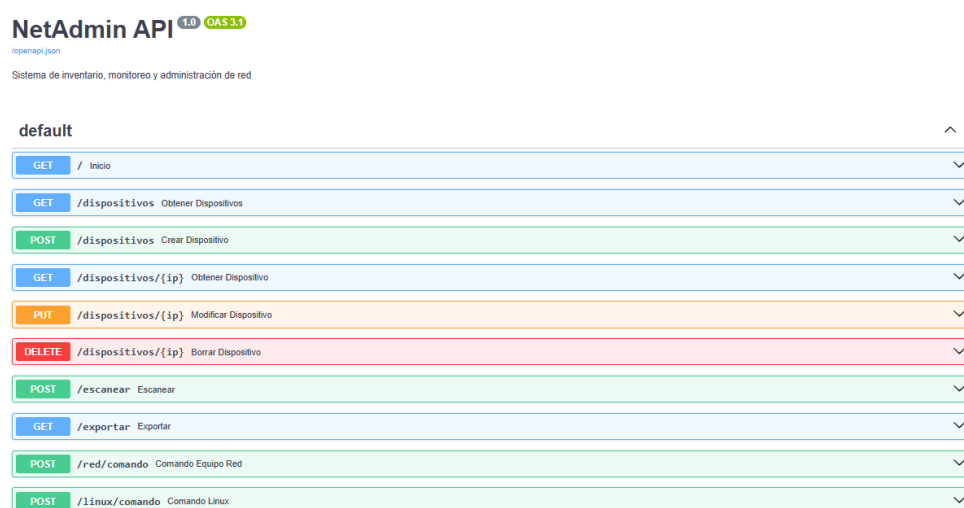


Figura 4. Documentación automática de la API en /docs.

14. Repositorio inicial del proyecto

El proyecto fue almacenado en GitHub para mantener un control de versiones y facilitar el trabajo colaborativo del equipo. En el repositorio se incluyeron los archivos principales del sistema, la carpeta app, la carpeta data, las pruebas unitarias, el archivo README.md, el archivo .gitignore y las dependencias del proyecto.

El repositorio permite consultar el código fuente, revisar los commits realizados y documentar los avances del equipo durante el desarrollo de la práctica.

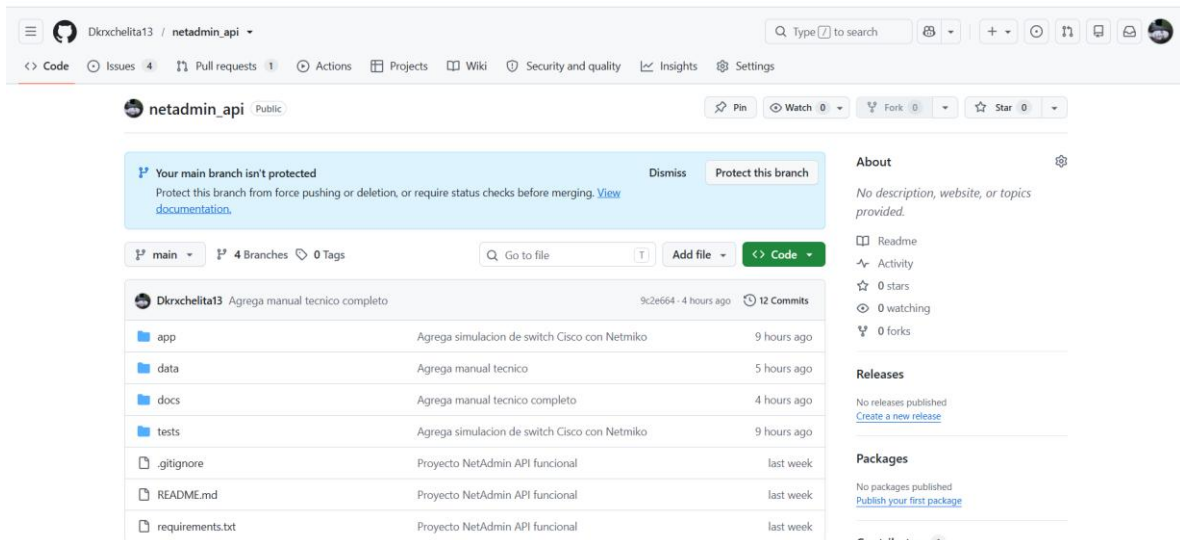


Figura 5. Repositorio NetAdmin API publicado en GitHub.

15. Desarrollo de la API REST

La API REST fue desarrollada utilizando **FastAPI**, un framework de Python que permite construir servicios web de forma rápida, estructurada y documentada automáticamente. El archivo principal del sistema es app/main.py, donde se definieron los endpoints encargados de recibir las solicitudes HTTP y devolver respuestas en formato JSON.

El sistema permite administrar un inventario de dispositivos de red mediante operaciones CRUD. Estas operaciones hacen posible registrar nuevos dispositivos, consultar el inventario completo, buscar un dispositivo por dirección IP, actualizar información existente y eliminar registros obsoletos.

La información de cada dispositivo se maneja mediante un modelo definido en `app/modelos.py`, el cual contiene los campos principales del inventario:

`ip`

`hostname`

`mac`

`tipo`

`sistema`

`estado`

Estos campos permiten representar información básica de un equipo de red, como su dirección IP, nombre de host, dirección MAC, tipo de dispositivo, sistema operativo aproximado y estado dentro del inventario.

16. Endpoints implementados

A continuación, se muestra la tabla de endpoints principales desarrollados para la administración del inventario.

Tabla 3. Endpoints CRUD implementados en NetAdmin API.

Método	Endpoint	Descripción
GET	/	Verifica que la API esté funcionando
GET	/dispositivos	Lista todos los dispositivos del inventario
GET	/dispositivos/{ip}	Busca un dispositivo por IP
POST	/dispositivos	Agrega un nuevo dispositivo
PUT	/dispositivos/{ip}	Actualiza un dispositivo existente
DELETE	/dispositivos/{ip}	Elimina un dispositivo del inventario
POST	/escanear	Escanea una red local o simulada
GET	/exportar	Exporta el inventario en JSON, YAML y XML
POST	/red/comando	Ejecuta comandos en switch/router mediante Netmiko
POST	/linux/comando	Ejecuta comandos en Linux mediante Paramiko

Estos endpoints fueron validados inicialmente desde la interfaz automática de FastAPI en `/docs` y posteriormente mediante Postman, donde se documentaron evidencias gráficas de cada operación. En las capturas del documento se observa la

creación, consulta, actualización y eliminación de dispositivos desde Swagger y Postman.

17. Creación de dispositivos con POST

La operación de creación de dispositivos se realizó mediante el endpoint:

POST /dispositivos

Este endpoint recibe un cuerpo en formato JSON con los datos del dispositivo que se desea agregar al inventario. Para la prueba se registró un dispositivo tipo router con la siguiente información:

```
{  
  "ip": "192.168.1.1",  
  "hostname": "router-principal",  
  "mac": "00:11:22:33:44:55",  
  "tipo": "router",  
  "sistema": "Cisco IOS",  
  "estado": "activo"  
}
```

Al enviar la solicitud correctamente, la API respondió con el código **201 Created**, indicando que el recurso fue creado exitosamente. La respuesta incluyó un mensaje de confirmación y los datos del dispositivo registrado.

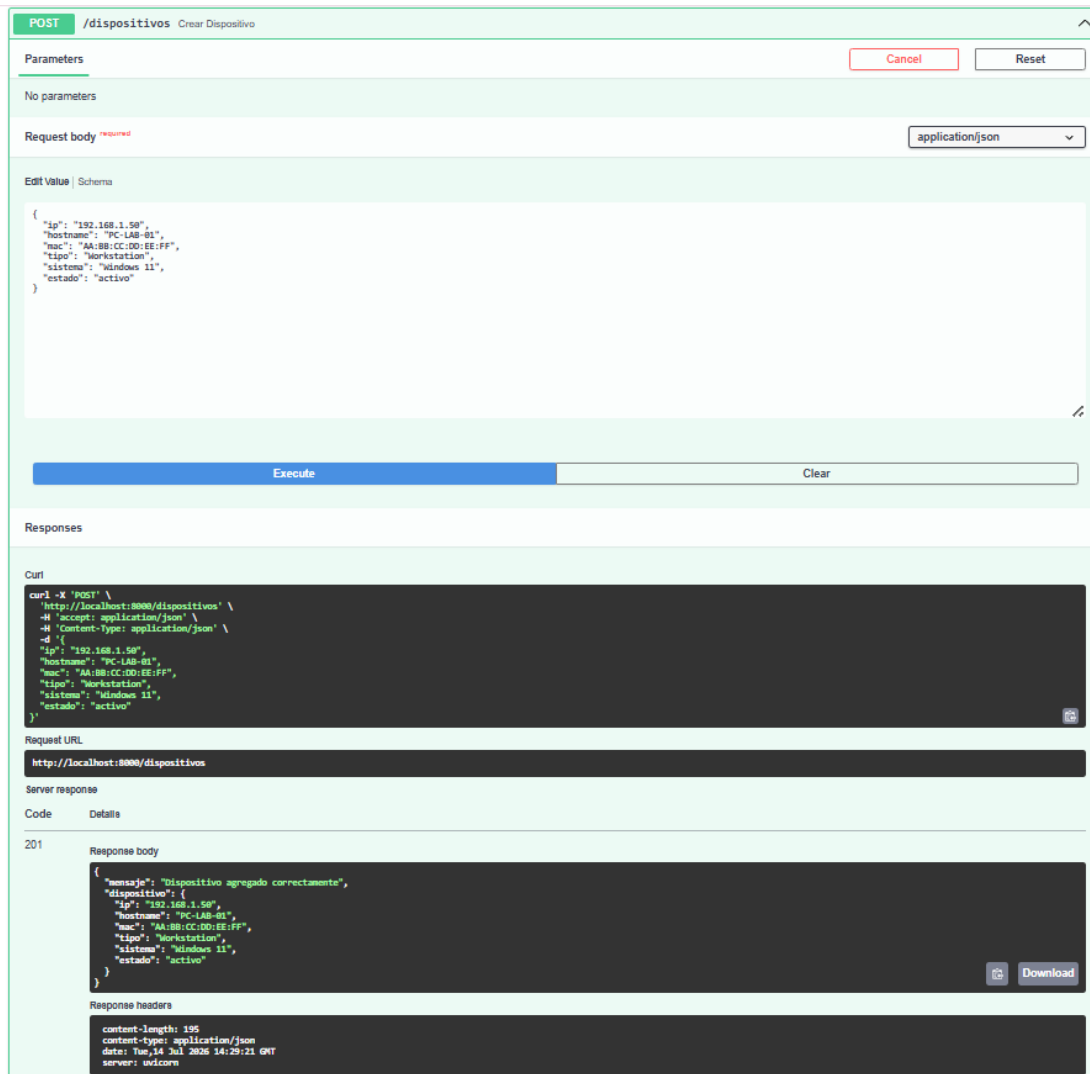


Figura 6. Creación de dispositivo mediante el endpoint POST /dispositivos.

18. Consulta del inventario con GET

Para consultar todos los dispositivos registrados, se utilizó el endpoint:

GET /dispositivos

Este endpoint devuelve una lista en formato JSON con todos los equipos almacenados en el inventario. Después de registrar el dispositivo manualmente, se ejecutó la consulta general y se verificó que el router agregado apareciera correctamente dentro de la respuesta.

Esta operación permitió comprobar que la API mantiene el inventario actualizado durante la ejecución del servidor.

The screenshot shows a REST client interface for the 'NetAdmin API' with the endpoint 'Listar dispositivos'. The request is a GET to 'http://127.0.0.1:8000/dispositivos'. The response is a 200 OK status with a 5 ms duration and 260 B of data. The response body is a JSON array containing one object with the following details:

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

```
1  [
2    {
3      "ip": "192.168.1.1",
4      "hostname": "router-principal",
5      "mac": "00:11:22:33:44:55",
6      "tipo": "router",
7      "sistema": "Cisco IOS",
8      "estado": "activo"
9    }
10 ]
```

Figura 7. Consulta del inventario actualizado mediante el endpoint GET /dispositivos.

19. Búsqueda de dispositivo por IP

Para consultar un dispositivo específico se utilizó el endpoint:

GET /dispositivos/{ip}

En la prueba se buscó el dispositivo con dirección IP:

192.168.1.1

La API localizó el registro dentro del inventario y devolvió la información correspondiente en formato JSON. Esta función es importante porque permite ubicar rápidamente un equipo específico sin necesidad de revisar todo el inventario.

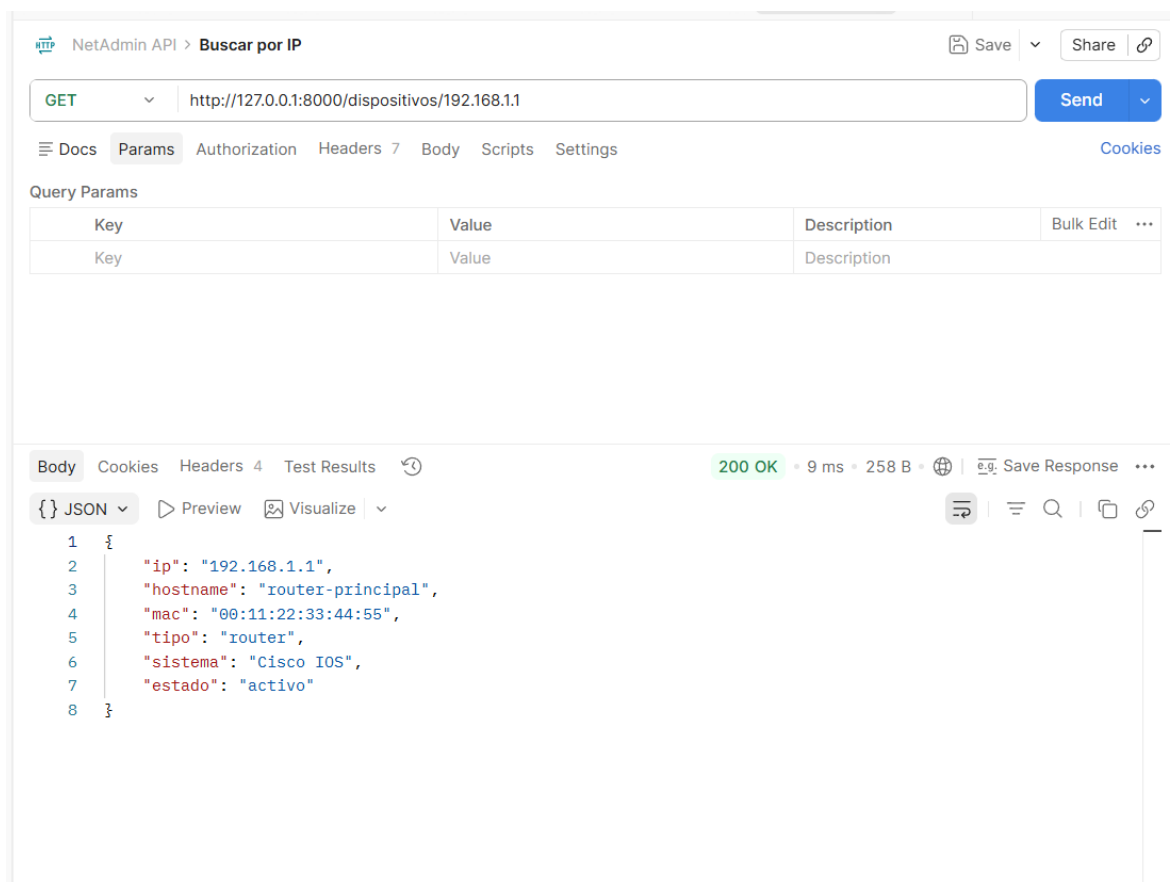


Figura 8. Búsqueda de dispositivo por dirección IP mediante GET /dispositivos/{ip}.

20. Actualización de dispositivos con PUT

La actualización de registros se realizó mediante el endpoint:

PUT /dispositivos/{ip}

Para esta prueba se actualizó la información del dispositivo con IP 192.168.1.1, modificando el hostname y el sistema operativo aproximado. El cuerpo JSON utilizado fue el siguiente:

```
{
```

```
"ip": "192.168.1.1",  
"hostname": "router-actualizado",  
"mac": "00:11:22:33:44:55",  
"tipo": "router",  
"sistema": "Cisco IOS XE",  
"estado": "activo"  
}
```

La API respondió con un mensaje indicando que el dispositivo fue actualizado correctamente. Esta operación demuestra que el inventario puede modificarse cuando cambian los datos de un equipo registrado.

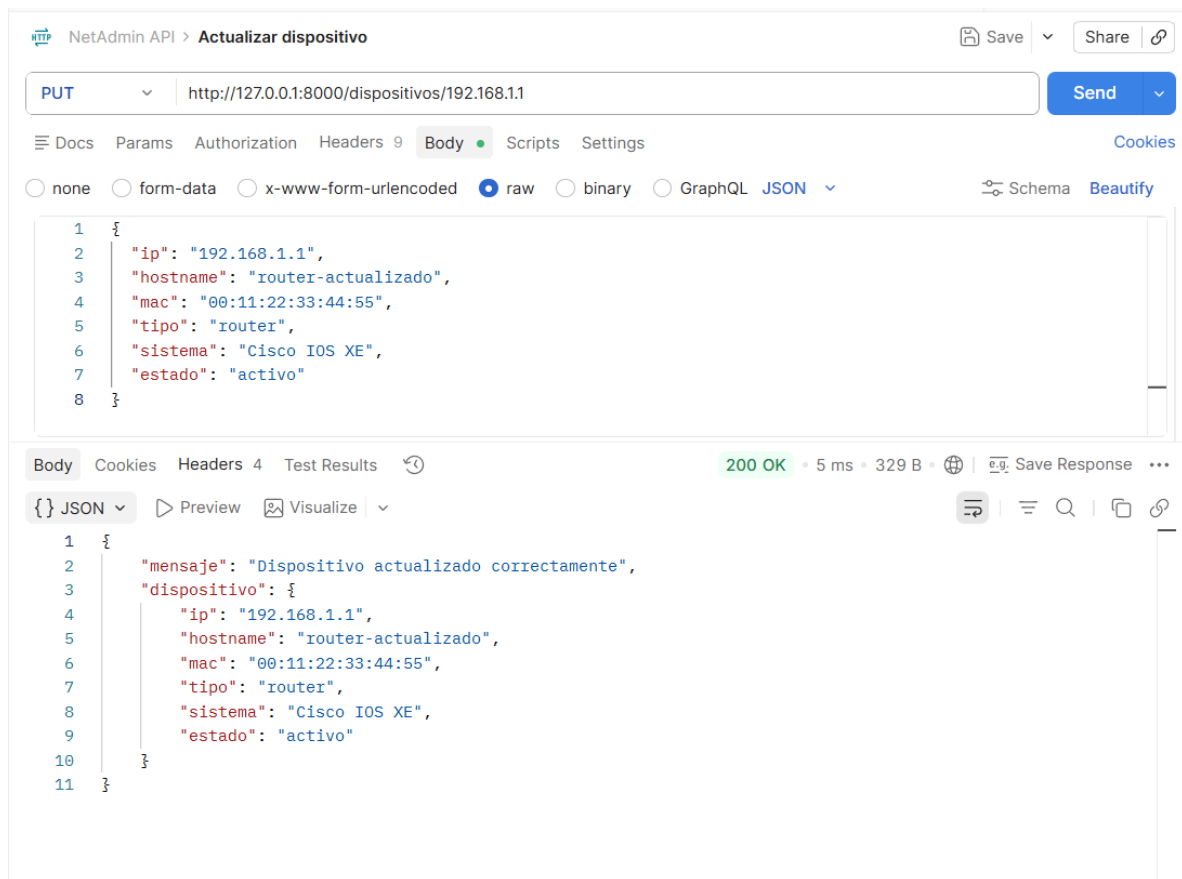


Figura 9. Actualización de dispositivo mediante el endpoint PUT /dispositivos/{ip}.

21. Eliminación de dispositivos con DELETE

Para eliminar registros obsoletos o de prueba se utilizó el endpoint:

DELETE /dispositivos/{ip}

En esta prueba se creó previamente un dispositivo de prueba con la IP:

192.168.163.50

Posteriormente, se eliminó mediante la solicitud:

DELETE /dispositivos/192.168.163.50

La API respondió con el mensaje:

```
{  
  "mensaje": "Dispositivo eliminado correctamente"  
}
```

Esta operación permite mantener limpio el inventario y retirar dispositivos que ya no forman parte de la red.

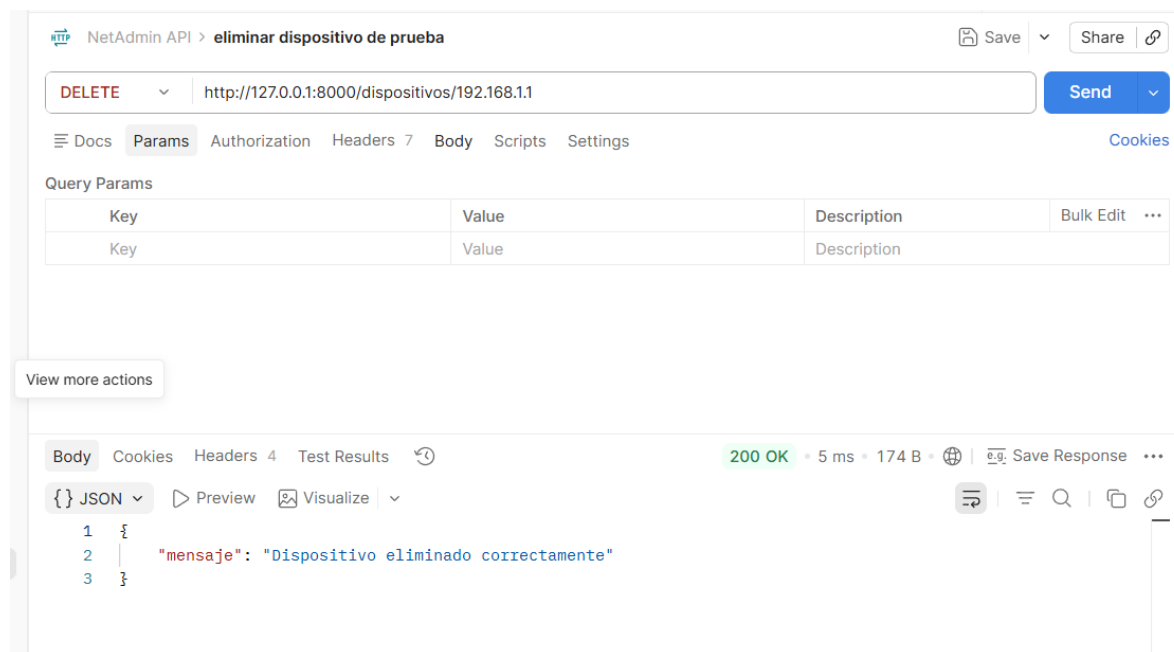


Figura 10. Eliminación de dispositivo de prueba mediante DELETE /dispositivos/{ip}.

22. Validación de CRUD en Postman

Además de las pruebas realizadas desde Swagger, se utilizó **Postman** para validar el funcionamiento de las operaciones principales de la API. Esta herramienta permitió enviar solicitudes HTTP, visualizar los códigos de estado y revisar las respuestas JSON generadas por el sistema.

Las pruebas realizadas en Postman incluyeron:

GET /

POST /dispositivos

GET /dispositivos

GET /dispositivos/{ip}

PUT /dispositivos/{ip}

DELETE /dispositivos/{ip}

Estas pruebas permitieron confirmar que las operaciones CRUD funcionan correctamente y que la API responde con códigos HTTP adecuados, como **200 OK** para consultas exitosas y **201 Created** para la creación de dispositivos.

My Collection > Get data Save Share

GET http://127.0.0.1:8000/ Send

Docs Params Authorization Headers 7 Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body 200 OK • 63 ms • 177 B • Save Response

{} JSON Preview Visualize

```
1 {
2   "mensaje": "NetAdmin API funcionando correctamente"
3 }
```

Figura 11. Verificación de funcionamiento de NetAdmin API desde Postman con GET /.

NetAdmin API > Post crear dispositivo Save Share

POST ▼ Send ▼

Docs Params Authorization Headers 9 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body ▼ 🔄 201 Created • 14 ms • 326 B • 🌐 Save Response ...

{} JSON ▼ ▶ Preview 🖨 Visualize ▼ ↩ ≡ 🔍 📄 🔗

```
1  {
2    "mensaje": "Dispositivo agregado correctamente",
3    "dispositivo": {
4      "ip": "192.168.1.1",
5      "hostname": "router-principal",
6      "mac": "00:11:22:33:44:55",
7      "tipo": "router",
8      "sistema": "Cisco IOS",
9      "estado": "activo"
10   }
11 }
```

Figura 12. Creación de dispositivo desde Postman con POST /dispositivos.

GET

/dispositivos

Obtener Dispositivos

Parameters

No parameters

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8000/dispositivos' \
  -H 'accept: application/json'
```

Request URL

http://localhost:8000/dispositivos

Server response

Code	Details
200	Response body <pre>{ "ip": "192.168.1.50", "hostname": "PC-LAB-01", "mac": "A4:8B:C4:D0:EE:FF", "tipo": "Workstation", "sistema": "Windows 11", "estado": "activo" }</pre> Response headers <pre>content-length: 134 content-type: application/json date: Tue, 14 Jul 2026 14:33:44 GMT server: unicorn</pre>

Responses

Code	Description	Links
200	Successful Response Media type application/json Controls Accept header. Example Value Schema "string"	No links

Figura 13. Listado de dispositivos desde Swagger con GET /dispositivos.

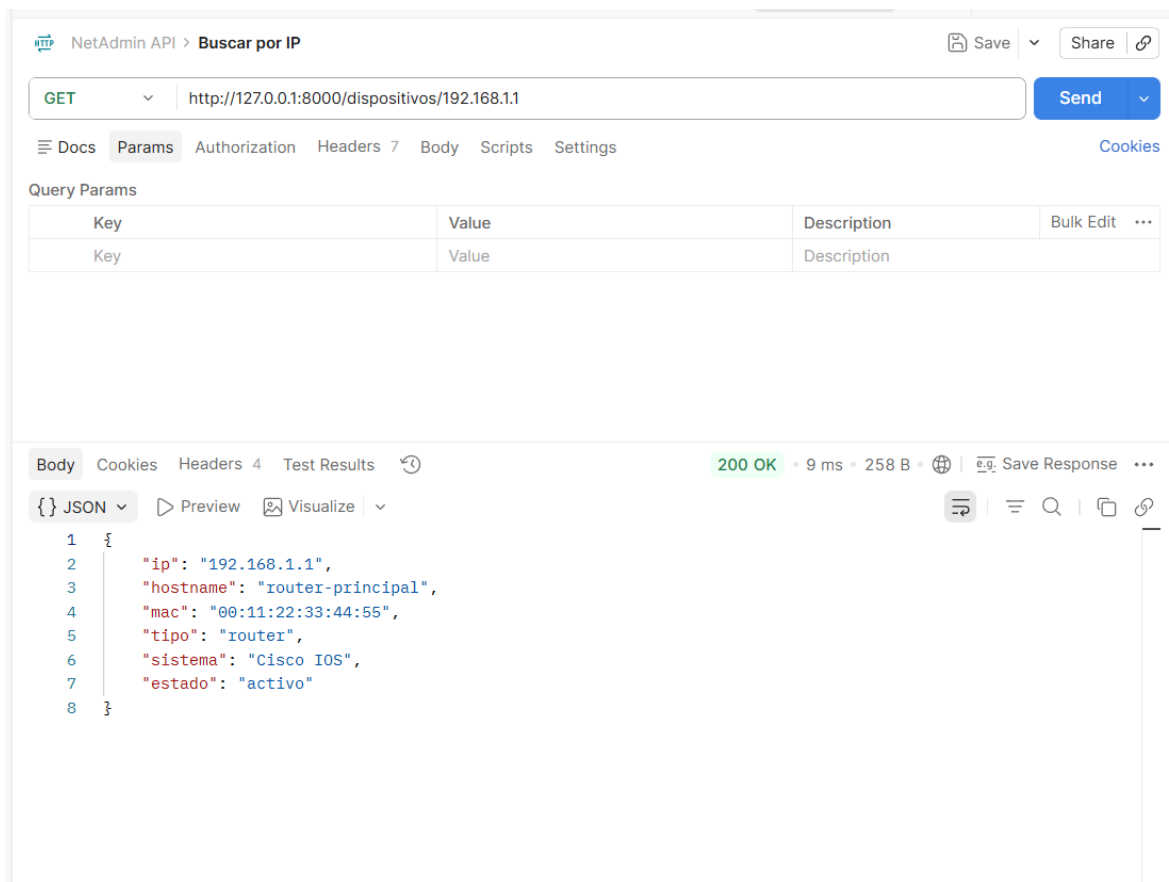


Figura 14. Búsqueda de dispositivo por IP desde Postman con GET /dispositivos/{ip}.

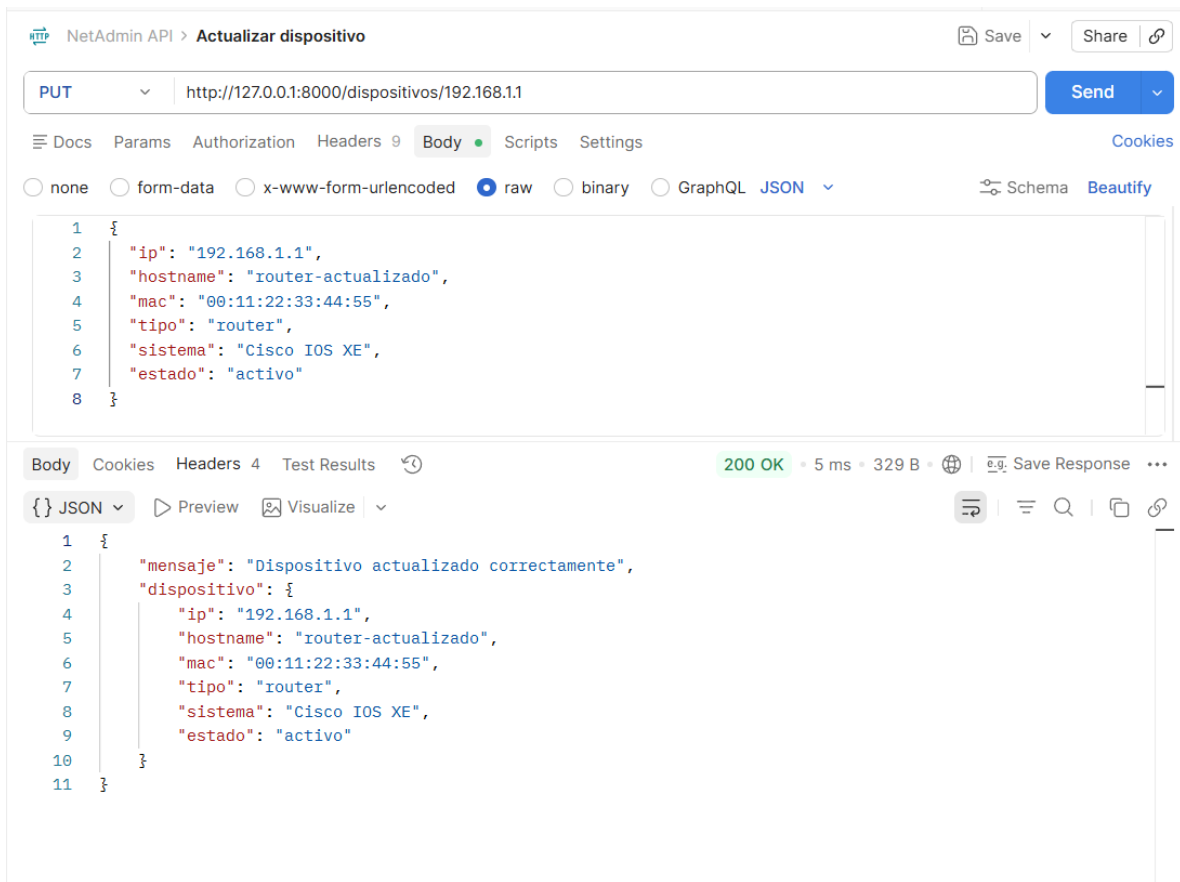


Figura 15. Actualización de dispositivo desde Postman con PUT `/dispositivos/{ip}`.

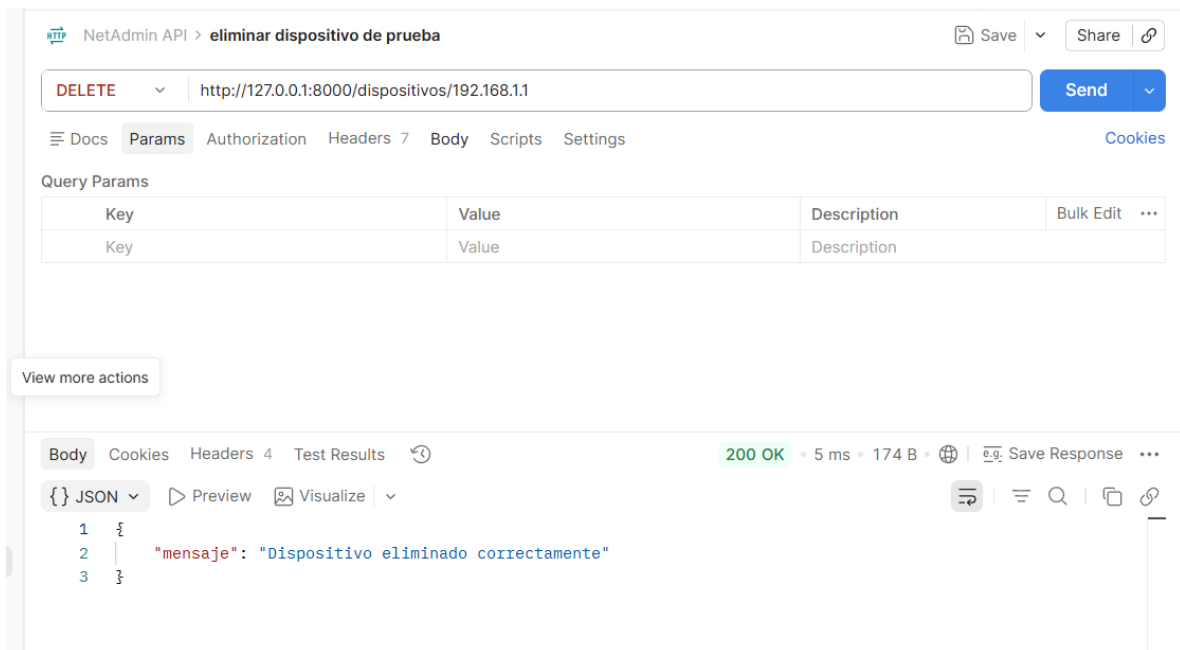


Figura 16. Eliminación de dispositivo desde Postman con DELETE `/dispositivos/{ip}`.

23. Resultado de la implementación CRUD

Con la implementación de los endpoints CRUD, **NetAdmin API** logró cumplir con una de las funciones principales solicitadas en la práctica: administrar dispositivos de red mediante una API REST. El sistema permite agregar dispositivos manualmente, consultar el inventario completo, buscar equipos por IP, modificar información existente y eliminar registros obsoletos.

Estas operaciones constituyen la base del sistema de inventario y permiten que posteriormente se integren funciones adicionales como escaneo de red, exportación de datos, administración remota y validación de errores HTTP.

24. Escaneo de red local simulada

Para cumplir con el requisito de detección de dispositivos activos, se implementó el endpoint:

POST /escanear

Este endpoint recibe como parámetro una red en formato CIDR y ejecuta un escaneo básico para identificar dispositivos activos. Para evitar afectar redes externas o no autorizadas, la prueba se realizó dentro de una red local simulada correspondiente al adaptador virtual de VMware.

La red utilizada fue:

192.168.163.0/28

Esta red fue seleccionada porque permite realizar una prueba controlada y rápida, revisando un rango reducido de direcciones IP. Al ejecutar el escaneo, la API detectó un dispositivo activo con la dirección IP 192.168.163.1, identificado con el hostname tzinacan y clasificado como router.

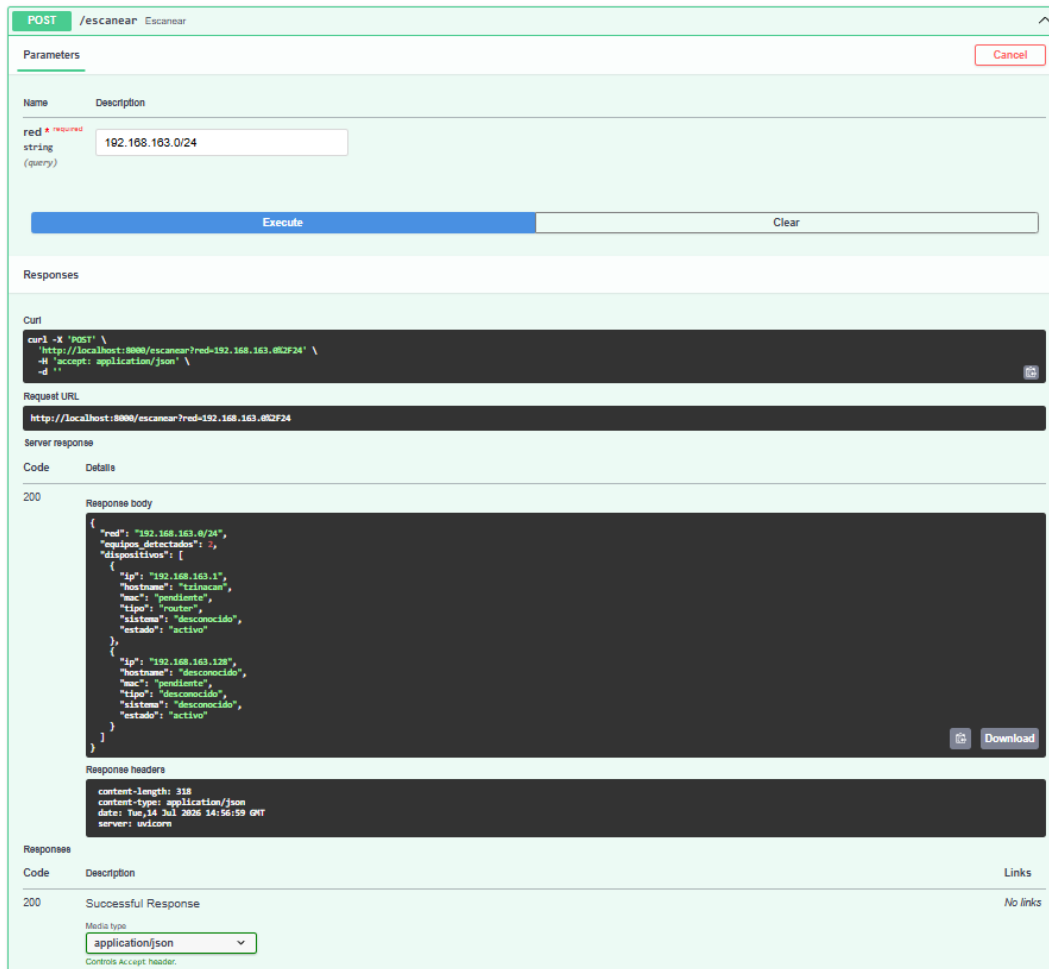


Figura 17. Escaneo de red local simulada mediante el endpoint POST /escanear swagger.

25. Resultado del escaneo

El resultado obtenido por la API fue una respuesta en formato JSON que incluyó la red analizada, el número de equipos detectados y la información de los dispositivos encontrados.

Ejemplo del resultado obtenido:

```
{  
  "red": "192.168.163.0/28",  
  "equipos_detectados": 1,
```

```
"dispositivos": [  
  {  
    "ip": "192.168.163.1",  
    "hostname": "tzinacan",  
    "mac": "pendiente",  
    "tipo": "router",  
    "sistema": "desconocido",  
    "estado": "activo"  
  }  
]  
}
```

El campo equipos_detectados confirmó que el sistema encontró un dispositivo activo en la red simulada. Además, los datos obtenidos fueron agregados al inventario para que pudieran consultarse posteriormente mediante el endpoint GET /dispositivos.

26. Actualización del inventario después del escaneo

Después de realizar el escaneo, se consultó nuevamente el inventario mediante el endpoint:

GET /dispositivos

Esta consulta permitió verificar que el dispositivo detectado durante el escaneo fue agregado correctamente al inventario. La respuesta mostró los dispositivos registrados manualmente y los dispositivos detectados automáticamente por el sistema.

Esta funcionalidad es importante porque permite mantener un inventario actualizado sin depender únicamente del registro manual de equipos.

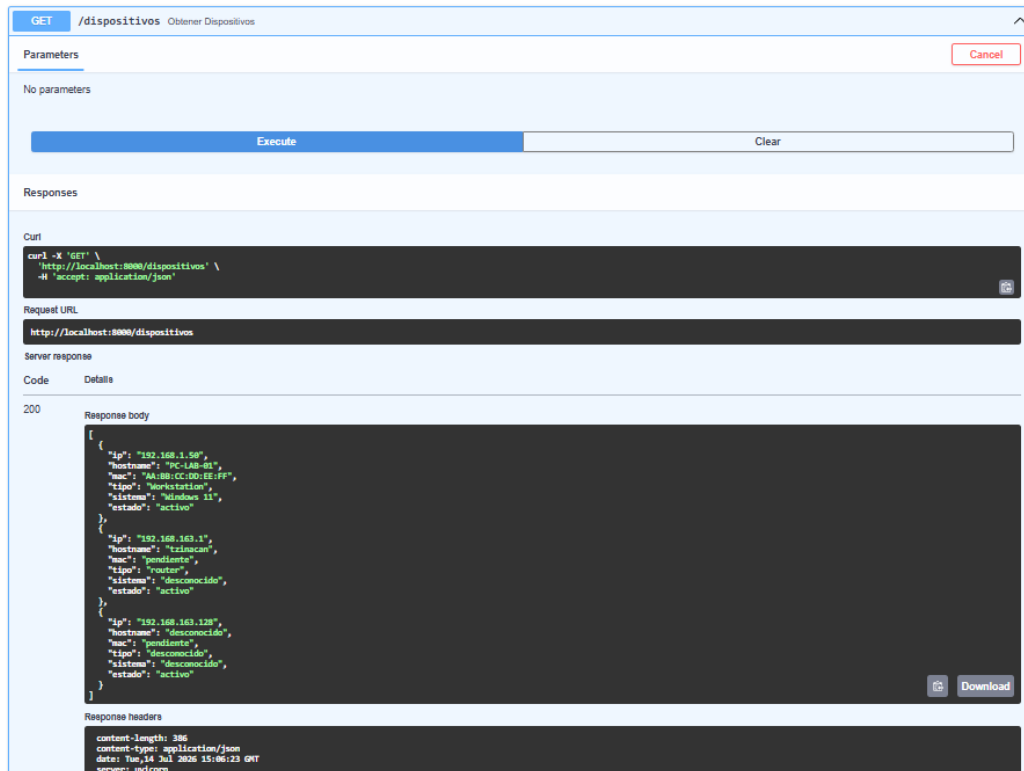


Figura 18. Consulta del inventario actualizado después del escaneo de red.

27. Exportación del inventario

Para cumplir con el requisito de guardar la información en formatos estructurados, se implementó el endpoint:

GET /exportar

Este endpoint toma los datos actuales del inventario y genera tres archivos dentro de la carpeta data:

inventario.json

inventario.yaml

inventario.xml

La exportación permite conservar la información de los dispositivos en formatos ampliamente utilizados para intercambio, documentación o integración con otros sistemas.

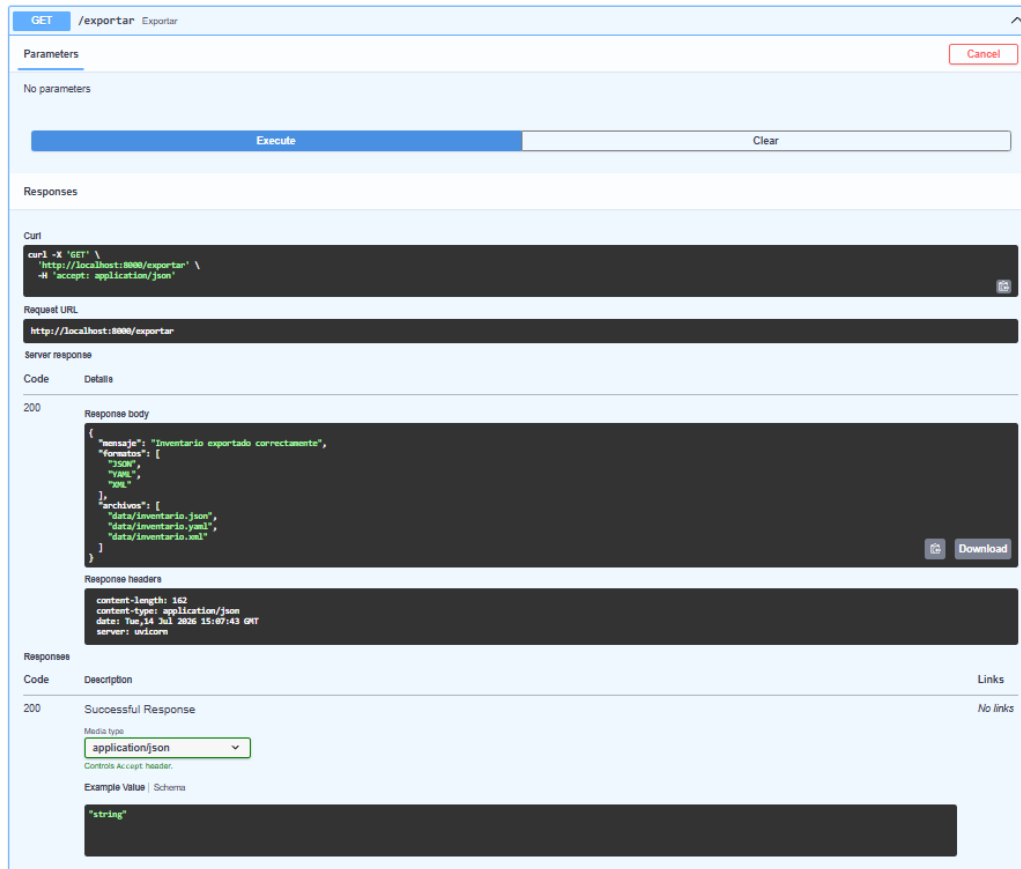


Figura 19. Exportación del inventario mediante el endpoint GET /exportar.

28. Archivos generados

Después de ejecutar el endpoint de exportación, se verificó la carpeta data del proyecto. En ella se observaron los archivos generados correctamente:

data/

└─ inventario.json

└─ inventario.yaml

└─ inventario.xml

Cada formato tiene una utilidad específica dentro del proyecto. JSON permite trabajar fácilmente con APIs y aplicaciones web, YAML facilita la lectura humana y XML permite representar información estructurada para sistemas que requieren este tipo de formato.

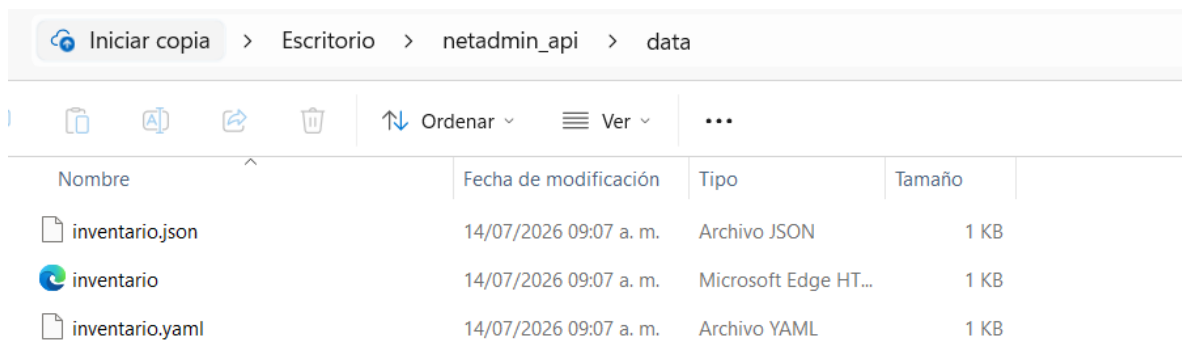


Figura 20. Archivos de inventario exportados en formato JSON, YAML y XML.

29. Formatos de exportación utilizados

Tabla 4. Formatos de exportación generados por NetAdmin API.

Formato	Archivo generado	Descripción
JSON	data/inventario.json	Formato ligero utilizado comúnmente en APIs y aplicaciones web.
YAML	data/inventario.yaml	Formato legible para humanos, útil para configuración y documentación.
XML	data/inventario.xml	Formato estructurado utilizado en integraciones con sistemas tradicionales.

30. Pruebas desde Postman

Además de Swagger, el escaneo y la exportación también fueron probados desde Postman. Esto permitió validar que los endpoints respondieran correctamente desde una herramienta externa de pruebas HTTP.

Las pruebas realizadas fueron:

POST `http://127.0.0.1:8000/escanear?red=192.168.163.0/28`

GET `http://127.0.0.1:8000/exportar`

En ambos casos se obtuvo una respuesta exitosa con código **200 OK**, confirmando que la API procesó correctamente las solicitudes.

NetAdmin API > Escanear red

POST http://127.0.0.1:8000/escanear?red=192.168.163.0/24

Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
✓ red	192.168.163.0/24		
Key	Value	Description	

Body Cookies Headers 4 Test Results

200 OK • 4 m 22.20 s • 444 B • Save Response

{ } JSON Preview Visualize

```
1 {
2   "red": "192.168.163.0/24",
3   "equipos_detectados": 2,
4   "dispositivos": [
5     {
6       "ip": "192.168.163.1",
7       "hostname": "tzinacan",
8       "mac": "pendiente",
9       "tipo": "router",
10      "sistema": "desconocido",
11      "estado": "activo"
12    },
13    {
14      "ip": "192.168.163.128",
15      "hostname": "desconocido",
16      "mac": "pendiente",
17      "tipo": "desconocido",
18      "sistema": "desconocido",
19      "estado": "activo"
20    }
21  ]
22 }
```

Globals Vault Tools

Figura 21. Escaneo de red desde Postman mediante POST /escanear.

NetAdmin API > Exportar inventario

GET http://127.0.0.1:8000/exportar

Send

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers 4 Test Results

200 OK • 27 ms • 288 B • Save Response

{ } JSON Preview Visualize

```
1 {
2   "mensaje": "Inventario exportado correctamente",
3   "formatos": [
4     "JSON",
5     "YAML",
6     "XML"
7   ],
8   "archivos": [
9     "data/inventario.json",
10    "data/inventario.yaml",
11    "data/inventario.xml"
12  ]
13 }
```

Figura 22. Exportación del inventario desde Postman mediante GET /exportar.

31. Resultado de la sección

Con la implementación del escaneo y la exportación, **NetAdmin API** cumple con los requisitos de detección de dispositivos activos, actualización del inventario y almacenamiento de información en formatos estructurados.

El sistema logró detectar un equipo activo en la red local simulada 192.168.163.0/28, agregarlo al inventario y exportar los datos generados en archivos JSON, YAML y XML. Estas funciones complementan las operaciones CRUD y permiten que el sistema funcione como una herramienta básica de inventario y monitoreo de red.

32. Administración remota en NetAdmin API

Además de las funciones de inventario, escaneo y exportación, el sistema **NetAdmin API** integra endpoints para ejecutar comandos remotos en equipos administrables. Para ello se utilizaron dos librerías principales: **Paramiko**, orientada a conexiones SSH con equipos Linux, y **Netmiko**, orientada a la administración de dispositivos de red como switches o routers Cisco.

Los endpoints implementados para esta funcionalidad fueron:

POST /linux/comando

POST /red/comando

El endpoint /linux/comando permite conectarse a un equipo Linux mediante SSH y ejecutar comandos como hostname, whoami o df -h. Por otra parte, el endpoint /red/comando permite ejecutar comandos tipo Cisco, como show ip interface brief y show vlan brief, utilizando Netmiko o una simulación controlada de switch Cisco.

33. Administración Linux mediante Paramiko

Para validar la administración remota con Paramiko, se utilizó una máquina Linux simulada con servicio SSH activo. El objetivo fue comprobar que la API pudiera conectarse a un equipo remoto, ejecutar un comando y devolver la salida en formato JSON.

Primero se verificó la dirección IP del servidor Linux simulado. En la práctica se utilizó una dirección dentro de la red virtual de VMware:

192.168.163.128

```
(kali@kali)-[~/HackingEtico-CTF-Explotacion]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:57:1a:5b brd ff:ff:ff:ff:ff:ff
    inet 192.168.163.128/24 brd 192.168.163.255 scope global dynamic noprefixroute eth0
        valid_lft 1630sec preferred_lft 1630sec
    inet6 fe80::dbb7:903d:1e9b:7740/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

Figura 23. Dirección IP del servidor Linux simulado para pruebas con Paramiko.

34. Activación del servicio SSH en Linux

Para que Paramiko pudiera establecer conexión remota, fue necesario habilitar el servicio SSH dentro de la máquina Linux. Se ejecutaron comandos para iniciar y verificar el estado del servicio:

```
sudo systemctl enable ssh
```

```
sudo systemctl start ssh
```

```
sudo systemctl status ssh
```

La salida mostró que el servicio **OpenSSH Server** se encontraba activo y escuchando en el puerto 22, lo cual permitió realizar pruebas de conexión desde Windows y desde la API.

```
(kali@kali)-[~]
└─$ sudo systemctl enable ssh
sudo systemctl start ssh
sudo systemctl status ssh
[sudo] password for kali:
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: disabled)
   Active: active (running) since Mon 2026-07-13 14:29:16 EDT; 53s ago
     Invocation: b6237174d26c4d17a09d20db9f128db2
       Docs: man:sshd(8)
             man:sshd_config(5)
    Main PID: 704 (sshd)
      Tasks: 1 (limit: 4440)
     Memory: 2M (peak: 2.8M)
        CPU: 35ms
    CGroup: /system.slice/ssh.service
            └─704 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Jul 13 14:29:16 kali systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
Jul 13 14:29:16 kali sshd[704]: Server listening on 0.0.0.0 port 22.
Jul 13 14:29:16 kali systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
Jul 13 14:29:16 kali sshd[704]: Server listening on :: port 22.
```

Figura 24. Servicio SSH activo en el servidor Linux simulado.

35. Prueba de conexión SSH desde Windows

Antes de ejecutar comandos desde la API, se realizó una prueba directa de conexión SSH desde Windows hacia el servidor Linux simulado. Esta prueba permitió confirmar que existía conectividad entre el equipo anfitrión y la máquina virtual.

El comando utilizado fue similar al siguiente:

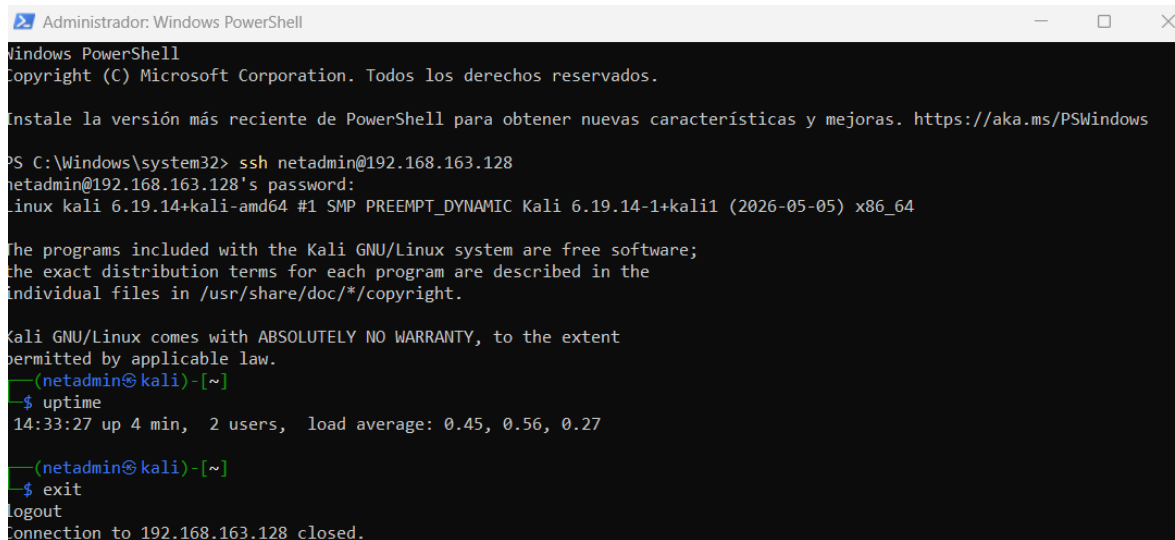
```
ssh netadmin@192.168.163.128
```

Después de iniciar sesión correctamente, se ejecutaron comandos básicos como:

```
hostname
```

```
uptime
```

La conexión fue exitosa, por lo que se confirmó que el servidor Linux estaba disponible para pruebas con Paramiko.



```
Administrador: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Windows\system32> ssh netadmin@192.168.163.128
netadmin@192.168.163.128's password:
Linux kali 6.19.14+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.19.14-1+kali1 (2026-05-05) x86_64

The programs included with the Kali GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
--(netadmin@kali)-[~]
--$ uptime
14:33:27 up 4 min,  2 users,  load average: 0.45, 0.56, 0.27

--(netadmin@kali)-[~]
--$ exit
logout
Connection to 192.168.163.128 closed.
```

Figura 25. Prueba de conexión SSH desde Windows hacia el servidor Linux simulado.

36. Ejecución de comando con Paramiko desde la API

Una vez confirmada la conexión SSH, se realizó la prueba desde Postman utilizando el endpoint:

POST `http://127.0.0.1:8000/linux/comando`

El cuerpo JSON enviado fue el siguiente:

```
{
  "ip": "192.168.163.128",
  "username": "netadmin",
  "password": "netadmin123",
  "comando": "hostname"
}
```

La API estableció conexión SSH mediante Paramiko, ejecutó el comando `hostname` y devolvió la salida en formato JSON. La respuesta exitosa confirmó que el endpoint `/linux/comando` funciona correctamente.

Ejemplo de respuesta:

```
{
  "ip": "192.168.163.128",
  "comando": "hostname",
  "salida": "kali\n"
}
```

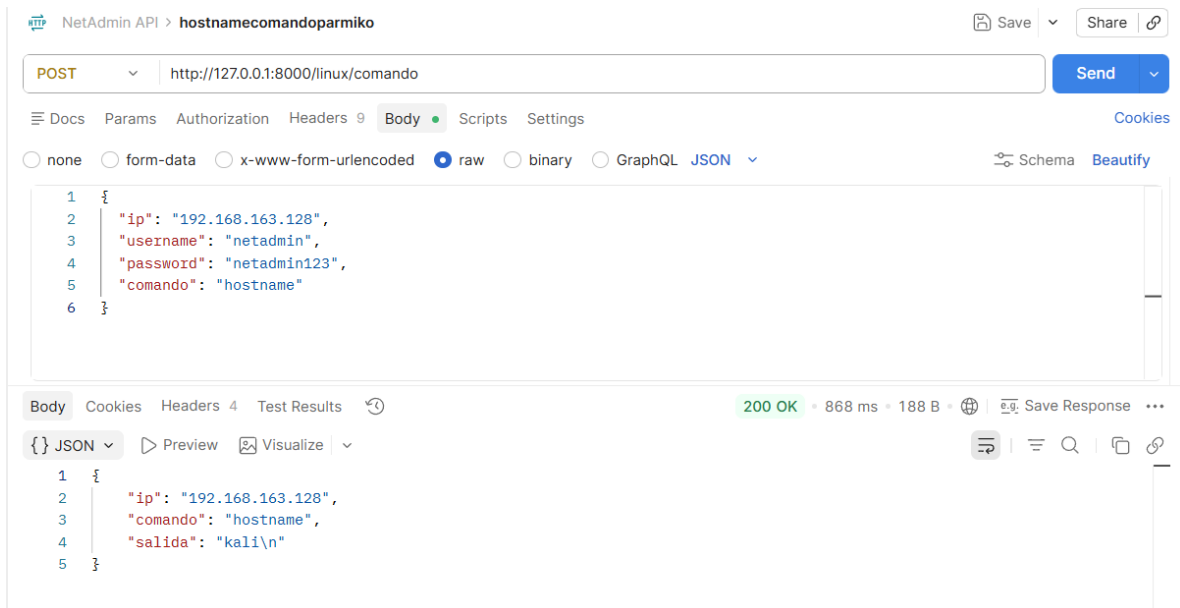


Figura 26. Ejecución del comando hostname en Linux mediante el endpoint /linux/comando con Paramiko.

37. Ejecución de comandos administrativos en Linux

Además del comando `hostname`, se realizaron pruebas adicionales para validar que Paramiko pudiera ejecutar otros comandos administrativos en el servidor Linux. Entre los comandos utilizados se encuentran:

`whoami`

`df -h`

El comando `whoami` permitió verificar el usuario conectado mediante SSH, mientras que `df -h` permitió obtener información del uso de disco del sistema Linux. La respuesta de la API mostró la salida de los comandos dentro del campo `salida`,

confirmando que el endpoint puede ejecutar instrucciones remotas y devolver los resultados al cliente.

NetAdmin API > hostnamecomandoparmiko

POST http://127.0.0.1:8000/linux/comando

Body

```
1 {
2   "ip": "192.168.163.128",
3   "username": "netadmin",
4   "password": "netadmin123",
5   "comando": "whoami"
6 }
```

200 OK • 544 ms • 190 B

Body

```
1 {
2   "ip": "192.168.163.128",
3   "comando": "whoami",
4   "salida": "netadmin\n"
5 }
```

NetAdmin API > hostnamecomandoparmiko

POST http://127.0.0.1:8000/linux/comando

Body

```
1 {
2   "ip": "192.168.163.128",
3   "username": "netadmin",
4   "password": "netadmin123",
5   "comando": "df -h"
6 }
```

200 OK • 570 ms • 715 B

Body

```
1 {
2   "ip": "192.168.163.128",
3   "comando": "df -h",
4   "salida": "Filesystem                Size      Used Avail Use% Mounted on
udev                                1.9G         0  1.9G   0% /
dev/ntmpfs                          391M    1.8M  390M   1% /run/ntmpfs
dev/sda1                            79G      38G   38G  50% /
ntmpfs                             2.0G      4.0K   2.0G   1% /dev/shm
ntmpfs                             2.0G      2.0G   2.0G   1% /tmp
ntmpfs                             1.0M         0  1.0M   0% /run/credentials/systemd-journald.
service/ntmpfs                     2.0G      12K   2.0G   1% /tmp/ntmpfs
getty@tty1.service/ntmpfs          391M    108K  391M   1% /run/user/1000/ntmpfs
1% /run/user/1000/ntmpfs
5 }
```

Figura 27. Ejecución de comandos administrativos en Linux mediante Paramiko.

38. Administración de switch/router mediante Netmiko

Para la validación de Netmiko se implementó una simulación de switch Cisco dentro del módulo `netmiko_admin.py`. Esta decisión permitió probar el endpoint `/red/comando` sin depender de un switch físico o de una imagen Cisco IOS real.

El switch simulado fue registrado en el inventario con los siguientes datos:

```
{  
  "ip": "192.168.163.10",  
  "hostname": "SW-NETADMIN",  
  "mac": "00:AA:BB:CC:DD:10",  
  "tipo": "switch",  
  "sistema": "Cisco IOS simulado",  
  "estado": "activo"  
}
```

La API respondió con código **201 Created**, confirmando que el switch simulado fue agregado correctamente al inventario.

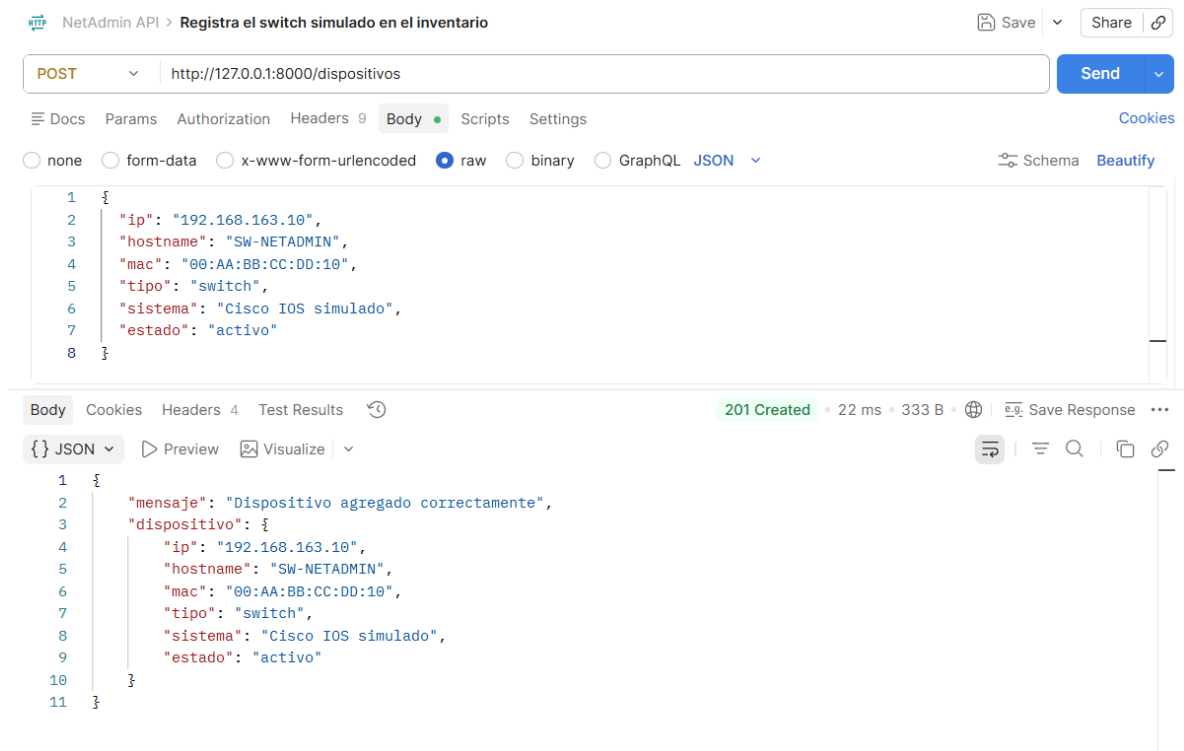


Figura 28. Registro del switch simulado en el inventario de NetAdmin API.

39. Ejecución de comando show ip interface brief

Para probar la administración de un equipo de red mediante Netmiko, se utilizó el endpoint:

POST `http://127.0.0.1:8000/red/comando`

El cuerpo JSON enviado fue:

```
{  "ip": "192.168.163.10",  "username": "admin",  "password": "cisco",  "secret": "class",  "device_type": "simulador_cisco",  "comando": "show ip interface brief"}
```

```
}
```

La API procesó la solicitud y devolvió una salida simulada similar a la que se obtendría en un switch Cisco real. La respuesta incluyó información de interfaces, direcciones IP, estado y protocolo.

Esta prueba permitió validar que el endpoint `/red/comando` recibe correctamente los datos del equipo, interpreta el comando solicitado y devuelve una salida estructurada al usuario.

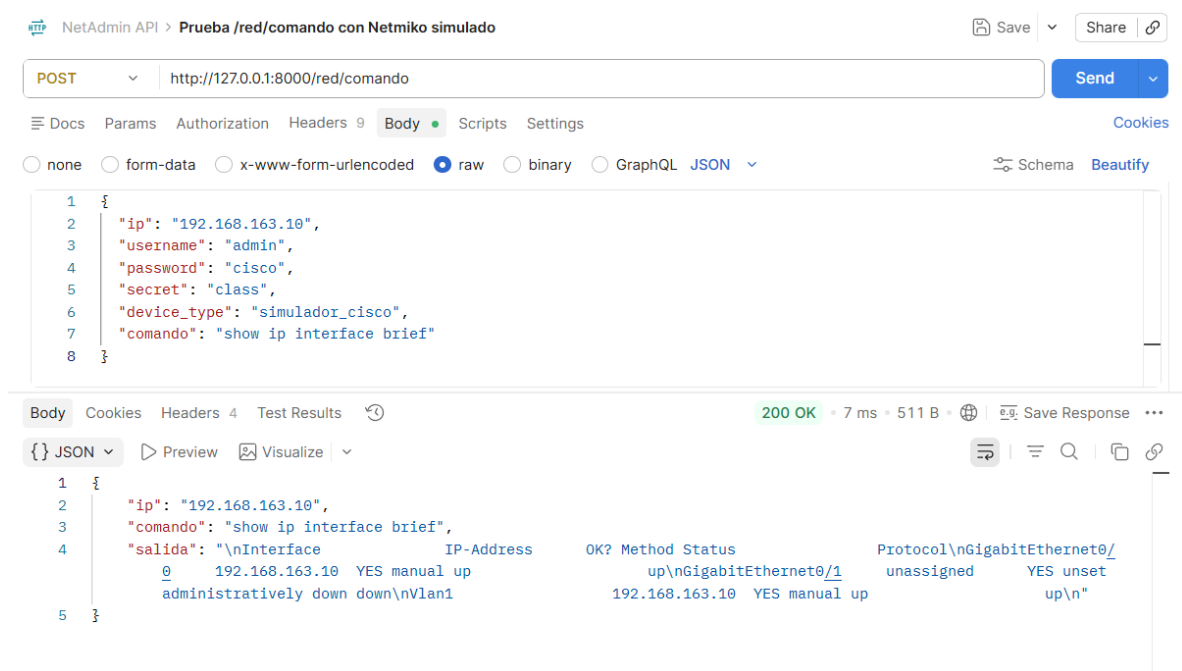


Figura 29. Ejecución del comando `show ip interface brief` en switch simulado mediante el endpoint `/red/comando`.

40. Consulta de VLANs mediante Netmiko

Como segunda prueba de Netmiko, se ejecutó el comando:

```
show vlan brief
```

El cuerpo JSON enviado fue:

```
{
```

```
  "ip": "192.168.163.10",
```

```
  "username": "admin",
```

```

"password": "cisco",

"secret": "class",

"device_type": "simulador_cisco",

"comando": "show vlan brief"
}

```

La API devolvió una salida simulada con información de VLANs, incluyendo VLAN por defecto y VLANs de administración o usuarios. Esta evidencia permitió demostrar que el sistema puede procesar distintos comandos de administración de red.

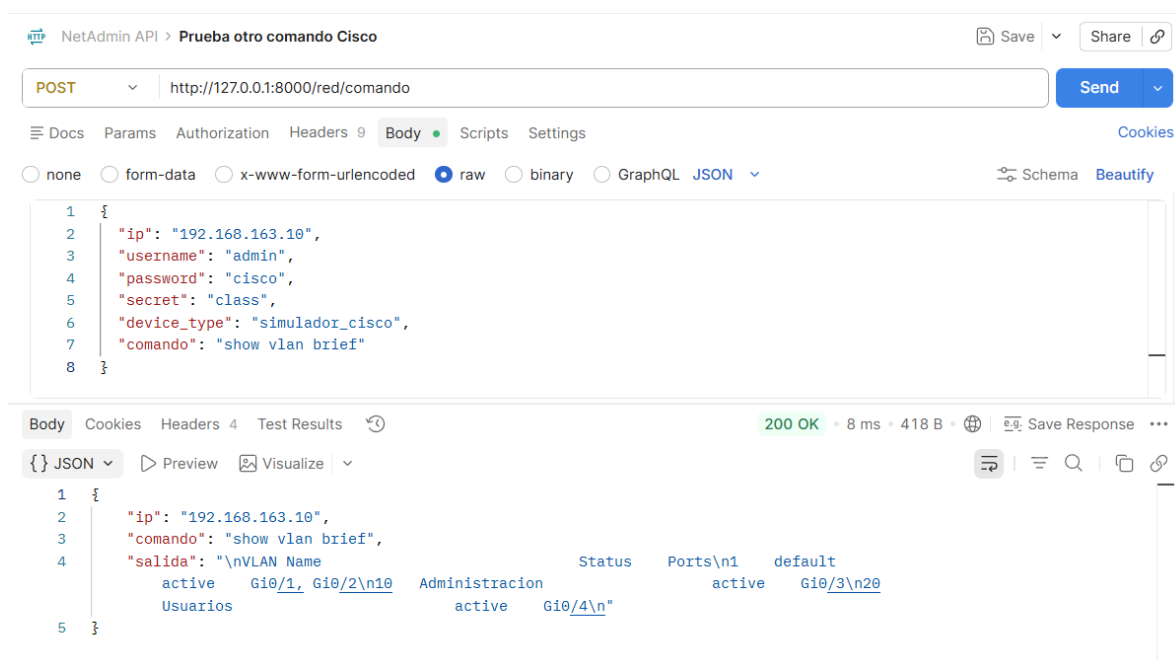


Figura 30. Consulta de VLANs en switch simulado mediante Netmiko.

41. Prueba unitaria del endpoint Netmiko simulado

Después de implementar la simulación de switch Cisco, se agregó una prueba unitaria para validar el endpoint `/red/comando`. Esta prueba permitió confirmar que la API respondiera correctamente al ejecutar el comando `show ip interface brief` usando el tipo de dispositivo `simulador_cisco`.

La ejecución de Pytest mostró que las pruebas fueron aprobadas correctamente, incluyendo la prueba del endpoint de Netmiko simulado.

```

• (venv) PS C:\Users\luzgr\Desktop\netadmin_api> python -m pytest
===== test session starts =====
platform win32 -- Python 3.12.0, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\luzgr\Desktop\netadmin_api
plugins: anyio-4.14.1
collected 6 items

tests\test_api.py ..... [100%]

===== 6 passed in 0.92s =====
○ (venv) PS C:\Users\luzgr\Desktop\netadmin_api>

```

Figura 31. Prueba unitaria del endpoint /red/comando con switch simulado.

42. Comparación entre Paramiko y Netmiko

Tabla 5. Comparación del uso de Paramiko y Netmiko en NetAdmin API.

Librería	Endpoint	Equipo administrado	Comandos probados	Resultado
Paramiko	/linux/comando	Servidor Linux simulado	hostname, whoami, df -h	Conexión SSH exitosa y salida en JSON
Netmiko	/red/comando	Switch Cisco simulado	show ip interface brief, show vlan brief	Salida simulada tipo Cisco en JSON

43. Resultado de la administración remota

Con las pruebas realizadas, se comprobó que **NetAdmin API** integra funcionalidades de administración remota para dos tipos de equipos. Por una parte, Paramiko permitió establecer conexión SSH con un servidor Linux simulado y ejecutar comandos reales sobre el sistema. Por otra parte, Netmiko permitió validar la lógica de administración de equipos de red mediante una simulación de switch Cisco.

Estas funciones amplían el alcance del sistema, ya que no solo permite registrar y consultar dispositivos, sino también ejecutar comandos administrativos sobre equipos remotos o simulados. Esto representa una base funcional para futuras mejoras, como la administración real de switches, routers, servidores Linux, respaldo de configuraciones o comparación de cambios.

44. Pruebas unitarias con Pytest

Para comprobar el funcionamiento básico de la API se implementaron pruebas unitarias utilizando **Pytest**. Estas pruebas permiten validar que los endpoints principales respondan correctamente sin necesidad de realizar cada operación manualmente desde Swagger o Postman.

Las pruebas se desarrollaron en el archivo:

tests/test_api.py

En este archivo se validaron funciones como la verificación de la API, creación de dispositivos, listado del inventario, búsqueda de dispositivos inexistentes y exportación del inventario. Posteriormente, se agregó una prueba adicional para comprobar el endpoint /red/comando con el switch Cisco simulado mediante Netmiko.

```
tests > test_api.py > test_comando_red_simulado
1 import os
2 import sys
3
4 sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), "..")))
5
6 from fastapi.testclient import TestClient
7 from app.main import app
8
9
10 client = TestClient(app)
11
12
13 def test_inicio():
14     response = client.get("/")
15     assert response.status_code == 200
16     assert response.json()["mensaje"] == "NetAdmin API funcionando correctamente"
17
18
19 def test_crear_dispositivo():
20     dispositivo = {
21         "ip": "10.0.0.1",
22         "hostname": "router-test",
23         "mac": "00:AA:BB:CC:DD:EE",
24         "tipo": "router",
25         "sistema": "Cisco IOS",
26         "estado": "activo"
27     }
28
29     response = client.post("/dispositivos", json=dispositivo)
30
31     assert response.status_code in [201, 400]
32
33
34 def test_listar_dispositivos():
35     response = client.get("/dispositivos")
36
37     assert response.status_code == 200
38     assert isinstance(response.json(), list)
39
40
41 def test_dispositivo_no_encontrado():
42     response = client.get("/dispositivos/255.255.255.255")
43
44     assert response.status_code == 404
45
46
47 def test_exportar_inventario():
48     response = client.get("/exportar")
49
50     assert response.status_code == 200
51     assert response.json()["mensaje"] == "Inventario exportado correctamente"
```

Figura 32. Código del archivo test_api.py para pruebas unitarias.

45. Ejecución de Pytest

Para ejecutar las pruebas unitarias se utilizó el entorno virtual del proyecto. Desde la carpeta principal de **NetAdmin API**, se ejecutó el siguiente comando:

```
python -m pytest
```

La ejecución de las pruebas mostró que los casos definidos fueron aprobados correctamente, lo cual confirma que la API responde de manera adecuada ante las operaciones evaluadas.

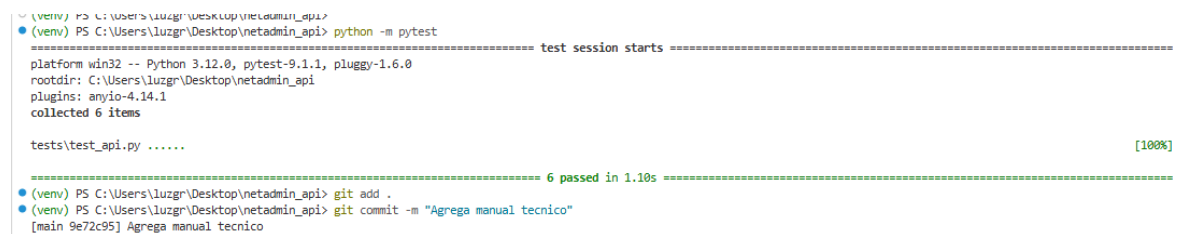
En la primera ejecución se obtuvo el resultado:

5 passed

Posteriormente, después de integrar la prueba del endpoint /red/comando con switch simulado, el resultado puede incrementarse a:

6 passed

Esto demuestra que las funciones principales del sistema fueron validadas de forma automatizada.



```
(venv) PS C:\Users\luzgr\Desktop\netadmin_api> python -m pytest
===== test session starts =====
platform win32 -- Python 3.12.0, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\luzgr\Desktop\netadmin_api
plugins: anyio-4.14.1
collected 6 items

tests\test_api.py ..... [100%]

===== 6 passed in 1.10s =====
(venv) PS C:\Users\luzgr\Desktop\netadmin_api> git add .
(venv) PS C:\Users\luzgr\Desktop\netadmin_api> git commit -m "Agrega manual tecnico"
[main 9e72c95] Agrega manual tecnico
```

Figura 33. Ejecución de Pytest usando Python del entorno virtual.

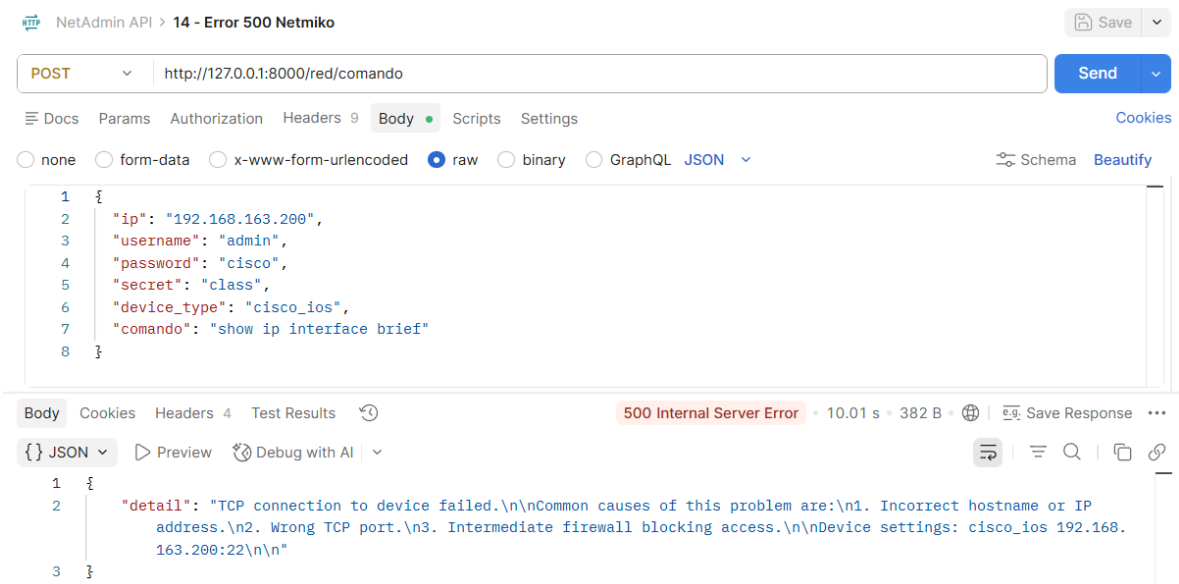


Figura 34. Resultado de pruebas unitarias incluyendo el endpoint /red/comando con switch simulado.

46. Importancia de las pruebas unitarias

Las pruebas unitarias permiten comprobar que los endpoints principales de la API funcionan correctamente antes de realizar la entrega del proyecto. En este caso, Pytest ayudó a validar que la API pudiera responder correctamente a solicitudes exitosas y a casos donde un dispositivo no existe en el inventario.

Además, estas pruebas facilitan la detección temprana de errores cuando se realizan cambios en el código. Por ejemplo, si se modifica un endpoint o una función del inventario, las pruebas permiten identificar si alguna funcionalidad dejó de operar correctamente.

Con esta validación, se refuerza la confiabilidad del sistema y se demuestra que **NetAdmin API** no solo fue probado manualmente desde Swagger y Postman, sino también mediante pruebas automatizadas.

47. Validación de códigos HTTP

Además de las pruebas unitarias, se realizó una validación de códigos de estado HTTP desde Postman. Esta validación permitió comprobar cómo responde la API ante

solicitudes correctas, creación de recursos, errores de validación, recursos inexistentes y fallos de conexión remota.

Los códigos evaluados fueron:

200 OK

201 Created

400 Bad Request

401 Unauthorized

403 Forbidden

404 Not Found

422 Unprocessable Entity

500 Internal Server Error

Los códigos **200** y **201** corresponden a operaciones exitosas. El código **400** se generó al intentar registrar una IP duplicada. El código **404** se obtuvo al consultar una IP inexistente. El código **422** fue generado automáticamente por FastAPI al enviar un cuerpo JSON incompleto. Finalmente, el código **500** se generó al provocar fallos de conexión en los endpoints de administración remota con Paramiko y Netmiko.

Los códigos **401** y **403** no fueron implementados en esta versión, ya que el sistema aún no cuenta con autenticación ni control de permisos. Sin embargo, se documentan como mejoras futuras para una versión con autenticación por token y roles de usuario.

48. Error 400: IP duplicada

El código **400 Bad Request** se generó al intentar registrar dos veces una misma dirección IP en el inventario. Esta validación evita que existan dispositivos duplicados dentro del sistema.

La prueba se realizó enviando nuevamente una solicitud al endpoint:

POST /dispositivos

con una IP que ya se encontraba registrada.

La API respondió con un mensaje indicando que la IP ya existía en el inventario.

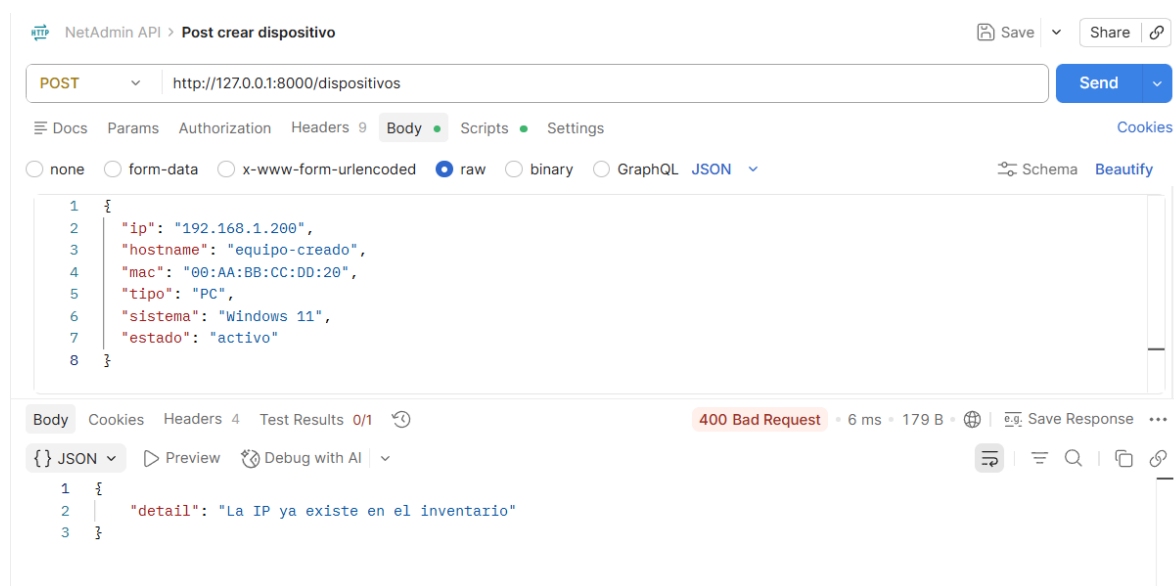


Figura 35. Error 400 generado al intentar registrar una IP duplicada.

49. Error 404: dispositivo no encontrado

El código **404 Not Found** se generó al buscar una dirección IP que no existía dentro del inventario. Para esta prueba se utilizó el endpoint:

GET `/dispositivos/10.10.10.10`

Como la IP consultada no estaba registrada, la API respondió con el mensaje:

```
{
  "detail": "Dispositivo no encontrado"
}
```

Esta validación permite informar al usuario cuando intenta consultar un recurso inexistente.

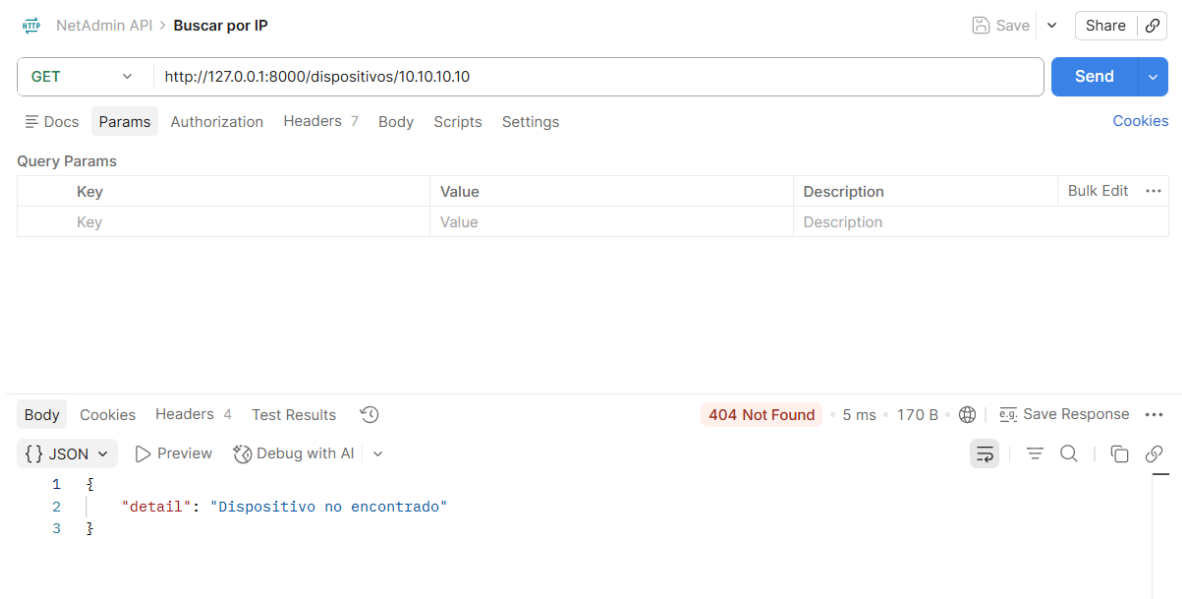


Figura 36. Error 404 generado al buscar un dispositivo inexistente.

50. Error 422: cuerpo JSON incompleto

El código **422 Unprocessable Entity** fue generado automáticamente por FastAPI al enviar un cuerpo JSON incompleto al endpoint de creación de dispositivos.

La prueba se realizó enviando una solicitud a:

POST /dispositivos

con un cuerpo JSON sin el campo obligatorio ip, por ejemplo:

```
{
  "hostname": "sin-ip",
  "mac": "00:00:00:00:00:00",
  "tipo": "PC",
  "sistema": "Windows",
  "estado": "activo"
}
```

FastAPI detectó que faltaba un campo requerido por el modelo Dispositivo y devolvió el código **422**. Esta respuesta ayuda a validar que los datos enviados por el usuario cumplan con la estructura definida en la API.

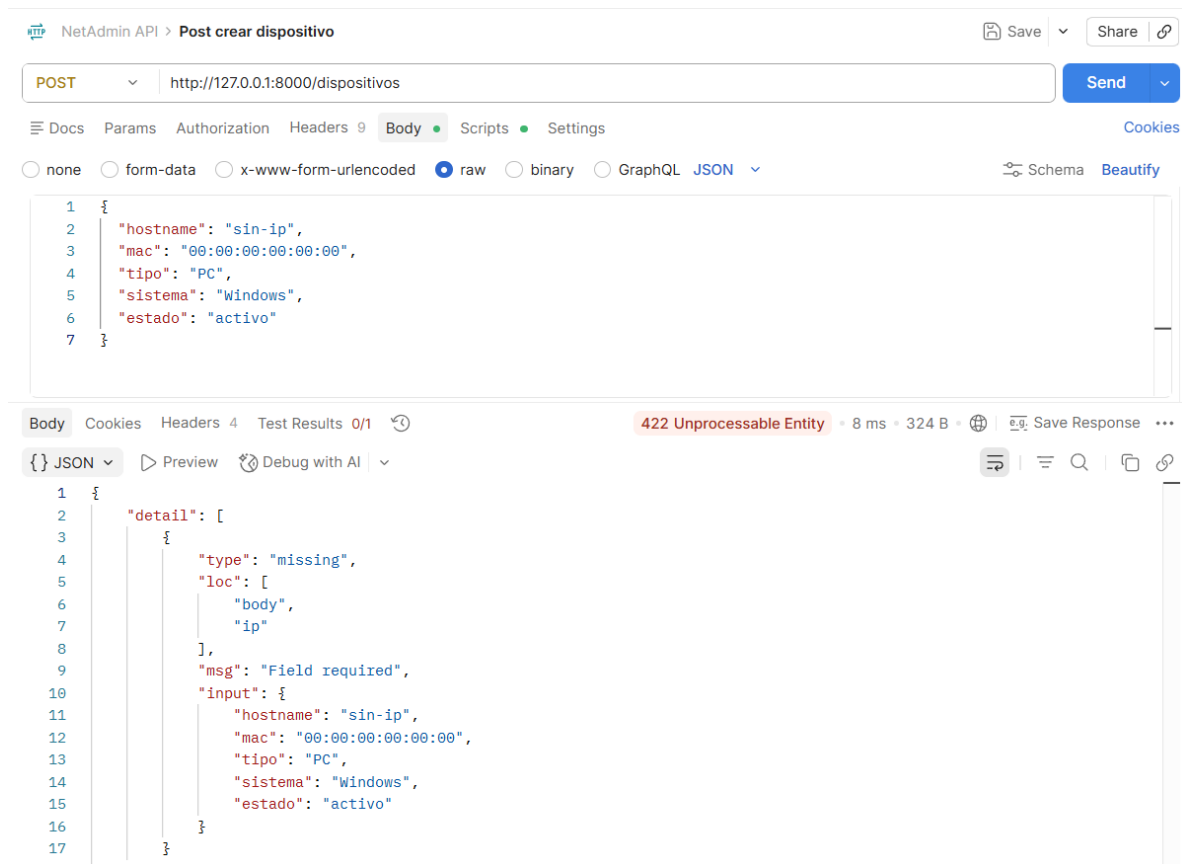


Figura 37. Error 422 generado al enviar un cuerpo JSON incompleto.

51. Error 500: fallo de conexión con Paramiko

El código **500 Internal Server Error** se generó al intentar ejecutar un comando remoto mediante Paramiko hacia una IP sin servicio SSH disponible o con credenciales no válidas.

La prueba se realizó utilizando el endpoint:

`POST /linux/comando`

con un cuerpo JSON similar al siguiente:

```
{
  "ip": "192.168.163.200",
```

```

"username": "usuario",

"password": "password",

"comando": "hostname"

}

```

Debido a que no existía una conexión SSH válida con ese equipo, la API devolvió un error controlado. Esta prueba permitió comprobar el comportamiento del sistema cuando Paramiko no logra establecer comunicación con el dispositivo remoto.

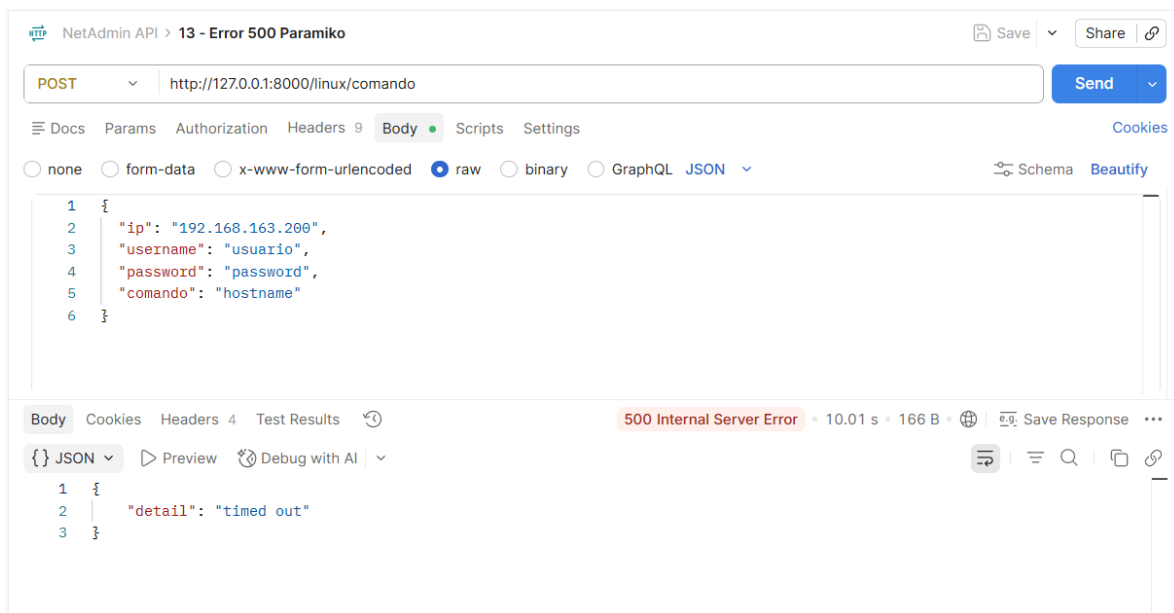


Figura 38. Error 500 generado por fallo de conexión en administración remota con Paramiko.

52. Error 500: fallo de conexión con Netmiko

También se generó un error **500 Internal Server Error** utilizando el endpoint de administración de equipos de red:

POST `/red/comando`

Para esta prueba se envió una solicitud con una IP sin un equipo Cisco real disponible:

```

{

  "ip": "192.168.163.200",

```

```
"username": "admin",

"password": "cisco",

"secret": "class",

"device_type": "cisco_ios",

"comando": "show ip interface brief"

}
```

Al no existir una conexión válida hacia un equipo administrable por Netmiko, la API respondió con un error interno controlado. Esta evidencia permitió comprobar el manejo de fallos en el módulo de administración de red.



Figura 39. Error 500 generado por fallo de conexión en administración remota con Netmiko.

53. Tabla de diagnóstico de códigos HTTP

La siguiente tabla resume los códigos HTTP evaluados durante las pruebas de la API, indicando su significado, la prueba realizada y la evidencia correspondiente.

Tabla 6. Diagnóstico de códigos HTTP evaluados en NetAdmin API.

Código HTTP	Estado	Descripción	Prueba realizada en NetAdmin API	Evidencia
-------------	--------	-------------	----------------------------------	-----------

200	OK	La solicitud fue procesada correctamente.	Consulta de dispositivos con GET /dispositivos, exportación con GET /exportar o escaneo con POST /escanear.	Figuras 11, 13, 21 y 22
201	Created	Se creó correctamente un nuevo recurso.	Registro de dispositivo mediante POST /dispositivos.	Figura 12
400	Bad Request	La solicitud no puede procesarse porque contiene datos incorrectos o repetidos.	Intento de registrar una IP duplicada en POST /dispositivos.	Figura 35
401	Unauthorized	Indica que el usuario no está autenticado.	No implementado en esta versión; se propone como mejora futura con autenticación por token.	No aplica
403	Forbidden	Indica que el usuario no tiene permisos suficientes.	No implementado en esta versión; se propone como mejora futura con roles y permisos.	No aplica
404	Not Found	El recurso solicitado no existe.	Búsqueda de una IP inexistente con GET /dispositivos/10.10.10.10.	Figura 36
422	Unprocessable Entity	El cuerpo JSON no cumple con el modelo requerido por la API.	Envío de un dispositivo sin el campo obligatorio ip.	Figura 37
500	Internal Server Error	Ocurrió un error interno durante la ejecución del proceso.	Fallo de conexión en /linux/comando o /red/comando usando IP sin servicio SSH o sin equipo Cisco disponible.	Figuras 38 y 39

54. Análisis de resultados de la validación HTTP

La validación de códigos HTTP permitió comprobar que **NetAdmin API** responde correctamente ante diferentes escenarios de uso. Las operaciones exitosas, como

consultar el inventario, exportar archivos o escanear una red, respondieron con código **200 OK**. La creación de dispositivos respondió con **201 Created**, confirmando que la API reconoce correctamente la generación de nuevos recursos.

En cuanto a los errores de cliente, el código **400 Bad Request** permitió validar la restricción de IP duplicada en el inventario. El código **404 Not Found** confirmó que la API informa adecuadamente cuando se busca un dispositivo inexistente. Además, el código **422 Unprocessable Entity** demostró que FastAPI valida automáticamente los datos enviados en el cuerpo JSON de la solicitud.

Finalmente, los errores **500 Internal Server Error** generados en Paramiko y Netmiko permitieron comprobar cómo responde el sistema ante fallos de conexión remota. Aunque estos errores representan fallos internos, fueron útiles para documentar el comportamiento de la API cuando no se cuenta con un equipo remoto disponible o credenciales válidas.

55. Resultado de la sección

Con las pruebas unitarias y la validación de códigos HTTP, se confirmó que **NetAdmin API** cuenta con mecanismos básicos de verificación funcional y manejo de errores. Las pruebas automatizadas con Pytest permitieron validar los endpoints principales, mientras que las pruebas en Postman permitieron observar el comportamiento del sistema ante solicitudes válidas e inválidas.

Esta sección demuestra que la API fue evaluada tanto en escenarios exitosos como en escenarios de error, cumpliendo con el requisito de validar códigos HTTP y documentar evidencias gráficas del funcionamiento del sistema. En tus capturas se observa la evidencia de Pytest, errores 400, 404, 422, 500 con Paramiko y 500 con Netmiko, además de la tabla de códigos HTTP elaborada para el reporte.

56. Uso de GitHub como repositorio del proyecto

Para organizar el desarrollo de **NetAdmin API** se utilizó **GitHub** como plataforma de control de versiones. El repositorio permitió almacenar el código fuente del proyecto, la documentación, las pruebas unitarias, archivos de configuración, evidencias y archivos exportados del inventario.

El uso de GitHub fue importante porque permitió mantener un historial de cambios durante el desarrollo, organizar las actividades del equipo y documentar el avance del proyecto mediante commits, ramas, issues y pull requests.

El repositorio incluyó los siguientes elementos principales:

app/

data/

docs/

tests/

README.md

requirements.txt

.gitignore

La carpeta app contiene el código principal de la API. La carpeta data almacena los archivos exportados del inventario. La carpeta docs contiene documentación técnica, como el manual técnico y la tabla de códigos HTTP. La carpeta tests contiene las pruebas unitarias del sistema.

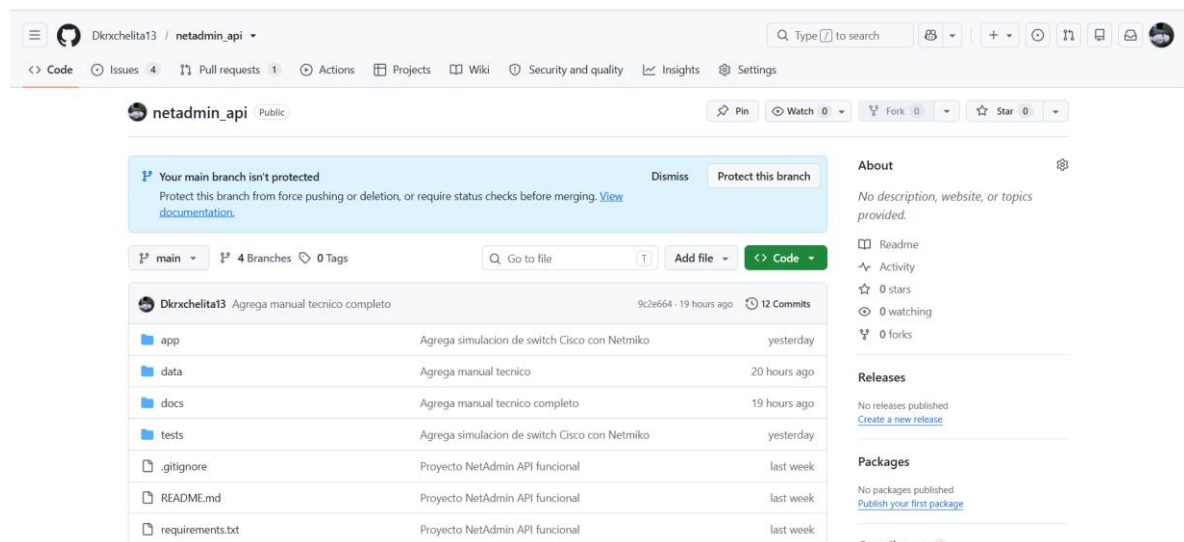


Figura 40. Repositorio NetAdmin API publicado en GitHub.

57. Organización del trabajo mediante issues

Para dividir las actividades del proyecto se utilizaron **issues** en GitHub. Esta herramienta permitió registrar tareas pendientes, asignar actividades y documentar el avance del equipo.

Entre las actividades organizadas mediante issues se consideraron las siguientes:

Realizar pruebas en Postman.

Elaborar manual técnico.

Integrar reporte final.

Agregar tabla de códigos HTTP.

Documentar evidencias de Paramiko y Netmiko.

El uso de issues permitió mantener una mejor organización del proyecto, ya que cada tarea quedó registrada dentro del repositorio y pudo ser revisada durante el desarrollo.

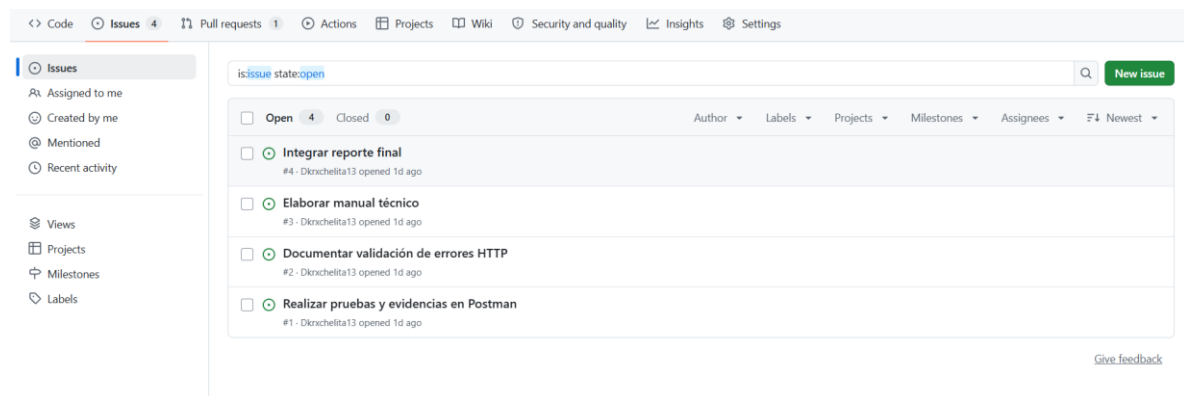


Figura 41. Issues creados en GitHub para organizar el trabajo colaborativo.

58. Uso de ramas de trabajo

Además de la rama principal main, se utilizaron ramas de trabajo para separar actividades específicas del proyecto. Esto permitió realizar cambios sin afectar directamente la versión principal del repositorio.

Algunas ramas utilizadas o propuestas para organizar el trabajo fueron:

feature/postman-evidencias

feature/manual-tecnico

feature/tabla-http

feature/reporte-final

Cada rama se utilizó para trabajar una parte específica del proyecto. Por ejemplo, la rama feature/manual-tecnico permitió agregar la documentación técnica, mientras que la rama feature/tabla-http permitió integrar la tabla de códigos HTTP.

Esta forma de trabajo facilita la revisión de cambios antes de integrarlos a la rama principal.

Default					
Branch	Updated	Check status	Behind / Ahead	Pull request	
main	19 hours ago		Default		...
Your main branch isn't protected Protect this branch from force pushing or deletion, or require status checks before merging. Learn more Dismiss <button>Protect this branch</button>					
Your branches					
Branch	Updated	Check status	Behind / Ahead	Pull request	
feature/postman-evidencias	19 hours ago		6 0	#8	...
feature/manual-tecnico	20 hours ago		8 0	#5	...
feature/tabla-http	yesterday		9 1	#6	...
Active branches					
Branch	Updated	Check status	Behind / Ahead	Pull request	
feature/postman-evidencias	19 hours ago		6 0	#8	...
feature/manual-tecnico	20 hours ago		8 0	#5	...
feature/tabla-http	yesterday		9 1	#6	...

Figura 42. Ramas de trabajo utilizadas para organizar el desarrollo del proyecto.

59. Pull request para integrar cambios

Una vez realizados los cambios en las ramas de trabajo, se utilizaron **pull requests** para integrarlos a la rama principal. El pull request permite revisar los cambios antes de fusionarlos, por lo que es una práctica común en proyectos colaborativos.

En el repositorio se creó un pull request para integrar evidencias y documentación al proyecto. Posteriormente, este fue fusionado correctamente en la rama main.

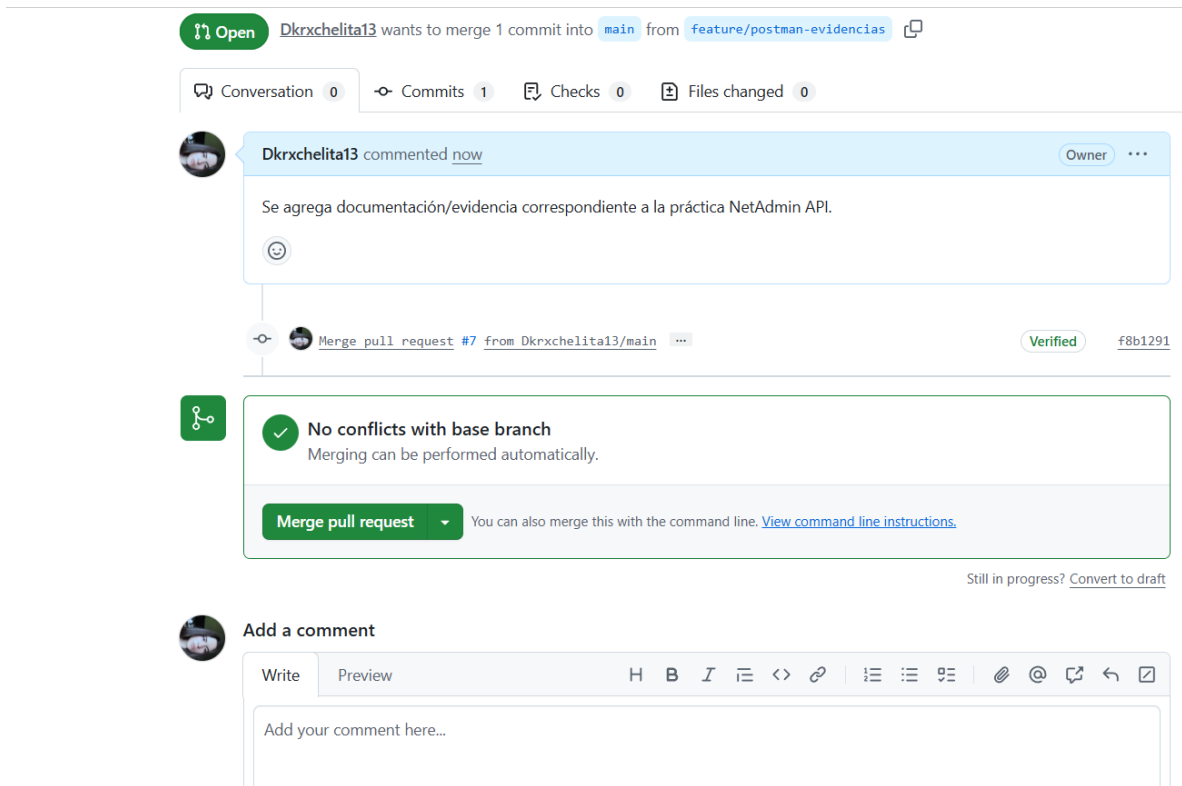


Figura 43. Pull request creado para integrar evidencias al proyecto.

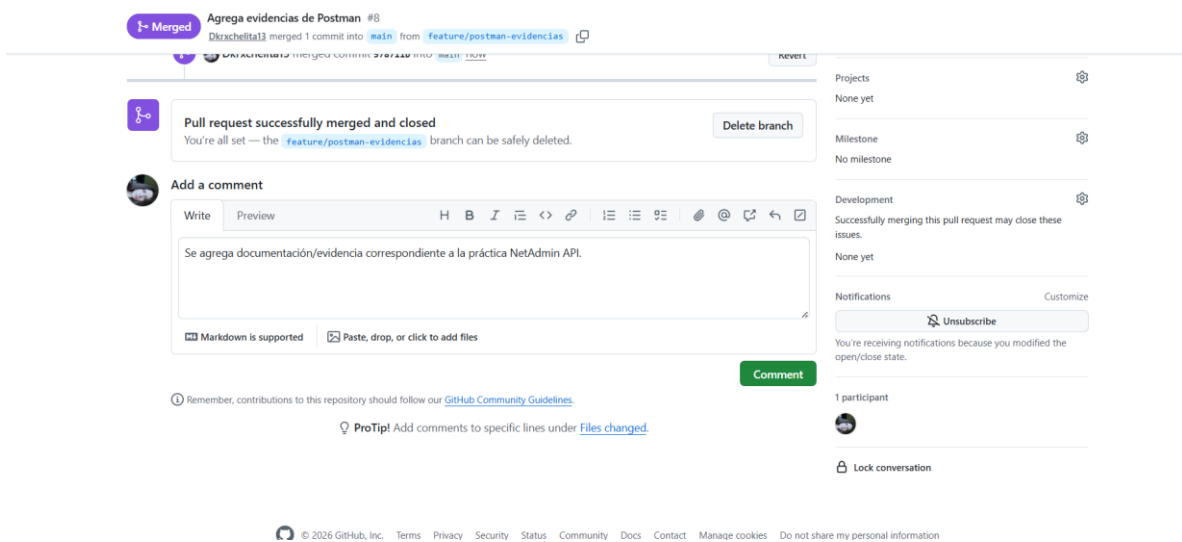


Figura 44. Pull request fusionado en la rama principal del repositorio.

60. Historial de commits

El historial de commits permite observar los cambios realizados durante el desarrollo del proyecto. Cada commit representa un avance o modificación registrada dentro del repositorio.

Durante el proyecto se realizaron commits relacionados con la integración del manual técnico, la tabla de códigos HTTP, la simulación de switch Cisco con Netmiko y las evidencias de Postman.

En la terminal se utilizó el siguiente comando para visualizar el historial:

```
git log --oneline --graph --all
```

Este comando permitió mostrar de forma gráfica los commits, ramas y fusiones realizadas durante el desarrollo del proyecto.

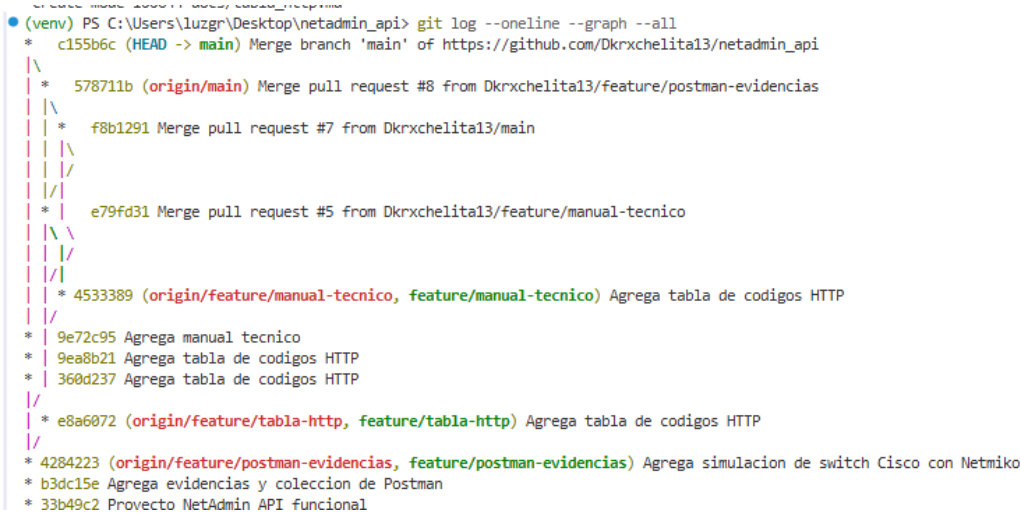


Figura 45. Historial de commits del proyecto NetAdmin API.

61. Evidencias de trabajo colaborativo

La siguiente tabla resume las evidencias utilizadas para demostrar el trabajo colaborativo en GitHub.

Tabla 7. Evidencias de trabajo colaborativo en GitHub.

Evidencia	Descripción	Figura relacionada
-----------	-------------	--------------------

Repositorio GitHub	Contiene el código fuente, documentación, pruebas y archivos del proyecto.	Figura 40
Issues	Permiten organizar tareas como pruebas, manual técnico y tabla HTTP.	Figura 41
Ramas de trabajo	Separan actividades específicas antes de integrarlas a main.	Figura 42
Pull request creado	Permite revisar cambios antes de fusionarlos.	Figura 43
Pull request fusionado	Confirma la integración de cambios en la rama principal.	Figura 44
Historial de commits	Muestra el avance registrado durante el desarrollo.	Figura 45

62. Resultados generales del proyecto

Con el desarrollo de **NetAdmin API** se logró implementar una herramienta funcional para inventario, monitoreo básico y administración de dispositivos de red. El sistema permitió registrar dispositivos manualmente, consultar el inventario, buscar equipos por dirección IP, actualizar registros y eliminar dispositivos de prueba.

Además, se implementó un escaneo de red local simulada utilizando la red 192.168.163.0/28, donde se detectó un dispositivo activo que posteriormente fue agregado al inventario. También se integró la exportación del inventario en tres formatos estructurados: JSON, YAML y XML.

El sistema fue probado desde Swagger y Postman, documentando evidencias de cada endpoint. Asimismo, se realizaron pruebas unitarias con Pytest para validar el funcionamiento de la API. También se implementaron pruebas de administración remota mediante Paramiko para equipos Linux y Netmiko para un switch Cisco simulado.

Finalmente, el proyecto fue versionado en GitHub, donde se documentaron ramas, issues, commits y pull requests como evidencia del trabajo colaborativo.

63. Cumplimiento de requisitos de la práctica

Tabla 8. Cumplimiento de requisitos del proyecto NetAdmin API.

Requisito solicitado	Implementación realizada	Estado
----------------------	--------------------------	--------

API REST para administración de dispositivos	Se implementaron endpoints CRUD con FastAPI.	Cumplido
Registro de dispositivos	Se implementó POST /dispositivos.	Cumplido
Consulta del inventario	Se implementó GET /dispositivos.	Cumplido
Búsqueda por IP	Se implementó GET /dispositivos/{ip}.	Cumplido
Actualización de dispositivos	Se implementó PUT /dispositivos/{ip}.	Cumplido
Eliminación de dispositivos	Se implementó DELETE /dispositivos/{ip}.	Cumplido
Escaneo de red	Se implementó POST /escanear.	Cumplido
Exportación en JSON, YAML y XML	Se implementó GET /exportar.	Cumplido
Pruebas en Postman	Se documentaron evidencias de los endpoints principales.	Cumplido
Pruebas unitarias	Se implementaron pruebas con Pytest.	Cumplido
Validación de códigos HTTP	Se documentaron códigos 200, 201, 400, 404, 422 y 500; 401 y 403 quedaron como mejora futura.	Cumplido parcialmente
Paramiko	Se validó administración Linux mediante /linux/comando.	Cumplido
Netmiko	Se validó administración de switch Cisco simulado mediante /red/comando.	Cumplido
GitHub colaborativo	Se utilizaron repositorio, issues, ramas, commits y pull requests.	Cumplido

64. Conclusiones

El desarrollo de **NetAdmin API** permitió aplicar conocimientos de programación en Python, administración de redes, desarrollo de APIs REST, pruebas de software y control de versiones. A través de FastAPI se construyó una aplicación funcional capaz de administrar un inventario básico de dispositivos de red mediante endpoints HTTP.

El sistema permitió realizar operaciones CRUD sobre dispositivos, ejecutar un escaneo básico de red local simulada y exportar la información del inventario en formatos JSON, YAML y XML. Esto cumple con el objetivo de almacenar y administrar información de red de forma estructurada.

Las pruebas realizadas en Swagger y Postman permitieron comprobar el funcionamiento de los endpoints principales. Además, la validación de códigos HTTP permitió observar el comportamiento de la API ante operaciones exitosas, errores de cliente, recursos inexistentes y fallos de conexión remota.

La integración de Paramiko y Netmiko amplió el alcance del sistema, ya que permitió agregar funciones de administración remota. Paramiko fue utilizado para ejecutar comandos en un servidor Linux simulado mediante SSH, mientras que Netmiko fue utilizado para validar comandos de administración en un switch Cisco simulado.

Finalmente, el uso de GitHub permitió organizar el proyecto mediante repositorio, issues, ramas, commits y pull requests. Esto demostró una forma de trabajo ordenada y colaborativa, además de facilitar la trazabilidad de los cambios realizados durante el desarrollo.

65. Recomendaciones y mejoras futuras

Aunque **NetAdmin API** cumple con los requisitos principales de la práctica, existen mejoras que podrían implementarse en versiones posteriores para hacerlo más completo y cercano a un sistema profesional.

Tabla 9. Recomendaciones y mejoras futuras para NetAdmin API.

Mejora propuesta	Descripción
Base de datos	Sustituir el inventario en memoria por SQLite, PostgreSQL o MySQL.
Autenticación	Implementar inicio de sesión y autenticación mediante tokens JWT.
Roles y permisos	Agregar perfiles de usuario para administrador, operador y consulta.
Dashboard web	Crear una interfaz gráfica para visualizar dispositivos, estado y estadísticas.
Escaneo avanzado	Integrar Nmap o Scapy para obtener puertos abiertos, MAC y sistema operativo aproximado.
Programación de escaneos	Permitir escaneos automáticos en horarios definidos.
Respaldo de configuraciones	Usar Netmiko para obtener configuraciones de routers y switches reales.
Logs de auditoría	Guardar historial de operaciones realizadas dentro de la API.

Reportes automáticos	Generar reportes en PDF o Excel con el estado del inventario.
Dockerización	Crear un contenedor Docker para facilitar el despliegue del sistema.

66. Anexos

Los anexos permiten complementar el reporte con evidencias, archivos y documentación adicional del proyecto.

Se proponen los siguientes anexos:

Anexo A. Código fuente principal de NetAdmin API.

Anexo B. Archivo requirements.txt.

Anexo C. Colección de Postman.

Anexo D. Manual técnico del proyecto.

Anexo E. Tabla de códigos HTTP.

Anexo F. Evidencias de Swagger y Postman.

Anexo G. Evidencias de Paramiko y Netmiko.

Anexo H. Evidencias de GitHub.

67. Cierre del reporte

Con base en las pruebas realizadas y evidencias documentadas, se concluye que **NetAdmin API** cumple con el propósito de funcionar como una herramienta básica para inventario, monitoreo y administración de red. El proyecto integra desarrollo backend con FastAPI, manejo de datos estructurados, pruebas automatizadas, validación HTTP y administración remota mediante librerías especializadas de Python.

El desarrollo de esta práctica permitió fortalecer habilidades técnicas relacionadas con redes, programación, APIs REST, pruebas de software y control de versiones. Además, el proyecto puede servir como base para futuras mejoras orientadas a un sistema más completo de administración de infraestructura de red.

Link del repositorio: https://github.com/Dkrxchelita13/netadmin_api